GOVERNMENT SKSJ TECHNOLOGICAL INSTITUTE

(Affiliated to Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A Project Report (Phase II) on

AI BASED IDENTITY ANOMALY DETECTION SYSTEM

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Urvashi Tanwar USN: 1SK23CS055

Hemanth Kumar K S USN: 1SK23CS020

Veenashree USN: 1SK23CS057

Vishwanath Sanapur USN: 1SK23CS059

Under the guidance of

Dr. Anitha AC

Department of Computer Science and Engineering

GOVERNMENT SKSJ TECHNOLOGICAL INSTITUTE, Bengaluru

2026–2027

GOVERNMENT SKSJ TECHNOLOGICAL INSTITUTE

(Affiliated to Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A Project Report (Phase II) on

AI BASED IDENTITY ANOMALY DETECTION SYSTEM

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Urvashi Tanwar USN: 1SK23CS055

Hemanth Kumar K S USN: 1SK23CS020

Veenashree USN: 1SK23CS057

Vishwanath Sanapur USN: 1SK23CS059

Under the guidance of

Dr. Anitha AC

Department of Computer Science and Engineering

GOVERNMENT SKSJ TECHNOLOGICAL INSTITUTE, Bengaluru

2026–2027

CERTIFICATE

This is to certify that the project report entitled “AI Based Identity Anomaly Detection System” submitted by Urvashi Tanwar (1SK23CS055), Hemanth Kumar K S (1SK23CS020), Veenashree (1SK23CS057), Vishwanath Sanapur (1SK23CS059) to the Visvesvaraya Technological University, Belagavi, in partial fulfilment for the award of the degree of Bachelor of Engineering in Computer Science and Engineering is a bona fide record of project work carried out by them under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

The project work has been examined and approved for the academic requirements prescribed for the said degree.

_____	_____
Signature of the Guide	Signature of the Head of the Department
Dr. Anitha AC	Department of Computer Science and Engineering
Department of Computer Science and Engineering	Government SKSJ Technological Institute

Name of the Examiners: 1. _____ 2. _____

Place: Bengaluru Date: _____

DECLARATION

We, Urvashi Tanwar, Hemanth Kumar K S, Veenashree, and Vishwanath Sanapur, students of Computer Science and Engineering, Government SKSJ Technological Institute, Bengaluru, hereby declare that the project work entitled “AI Based Identity Anomaly Detection System” has been independently carried out by us under the guidance of Dr. Anitha AC, Department of Computer Science and Engineering, and submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering during the academic year 2026–2027.

We further declare that this report has not been submitted, in part or full, to any other University or Institution for the award of any other degree or diploma. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Place: Bengaluru

Date: _____

Name	**USN**	**Signature**
Urvashi Tanwar	1SK23CS055	
Hemanth Kumar K S	1SK23CS020	
Veenashree	1SK23CS057	
Vishwanath Sanapur	1SK23CS059	

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. Anitha AC, Department of Computer Science and Engineering, for the valuable guidance, continuous encouragement, and constructive feedback provided throughout the course of this project. The discussions on evaluation design and the careful review of our results materially improved the quality of this work.

We are thankful to the Project Coordinator and the Head of the Department of Computer Science and Engineering for their support and for providing the necessary facilities to carry out this work. We also thank the Principal, Government SKSJ Technological Institute, for maintaining an academic environment in which this project could be completed.

We acknowledge the Los Alamos National Laboratory for releasing the Cyber-1 dataset and the open-source communities behind DuckDB, LightGBM, PyTorch, Flask, and Playwright. Their tools made a reproducible, end-to-end study possible.

Finally, we thank our families and friends for their patience and constant support throughout this endeavour.

— Urvashi Tanwar, Hemanth Kumar K S, Veenashree, Vishwanath Sanapur

ABSTRACT

This project presents an AI-based identity anomaly detection system for authentication logs. From LANL Cyber-1, 29.9 million events from 604 users over 58 days yield 21 behavioural features covering pair rarity, destination popularity, failure history, velocity, and time-of-day context. A LightGBM classifier uses an LSTM autoencoder reconstruction error as its twenty-first feature, and a user-level split prevents leakage. On 5.4 million held-out events the model achieves PR-AUC 0.3682, recall 0.5667, and F1 0.4866; on the full dataset it detects 598 of 702 red-team events (85.2%) with 269 false positives. A Flask dashboard supports live scoring, replay, alerts, and explanations.

Keywords: Identity anomaly detection; UEBA; LANL Cyber-1; LightGBM; LSTM autoencoder; behavioural analytics.

TABLE OF CONTENTS

CERTIFICATE.....i
DECLARATION.....ii
ACKNOWLEDGEMENT.....iii
ABSTRACT.....iv
LIST OF FIGURES.....vii
LIST OF TABLES.....viii
ABBREVIATIONS.....ix
1. INTRODUCTION.....1
1.1 Overview.....1
1.2 Problem Statement.....2
1.3 Objectives.....2
1.4 Scope and Limitations.....3
1.5 Methodology Overview.....3
1.6 Organization of the Report.....4
2. LITERATURE SURVEY.....5
2.1 Review Approach.....5
2.2 Existing Approaches.....5
2.3 Research Gaps.....8
2.4 Summary.....8
3. SYSTEM REQUIREMENTS SPECIFICATION.....9
3.1 Functional Requirements.....9
3.2 Non-Functional Requirements.....9
3.3 Hardware Requirements.....10
3.4 Software Requirements.....10
3.5 Dataset Requirements.....11
3.6 Assumptions and Constraints.....11
4. SYSTEM DESIGN.....12
4.1 System Architecture.....12
4.2 Data Acquisition and Slicing.....13
4.3 Feature Engineering.....14
4.4 Model Design.....15
4.5 Decision Logic.....17
4.6 Data and Storage Design.....17
4.7 Dashboard Design.....17
5. IMPLEMENTATION.....19
5.1 Technology Stack.....19
5.2 Module Description.....19
5.3 Key Algorithms.....22
5.4 User Interface Implementation.....23
5.5 Implementation Challenges.....28
5.6 Deployment and Execution.....29
6. TESTING.....30
6.1 Testing Strategy.....30
6.2 Test Environment.....30
6.3 Test Cases.....31
6.4 Automated Test Results.....31
6.5 Observations and Limitations.....32

7. RESULTS AND DISCUSSION.....33
7.1 Experimental Setup.....33
7.2 Held-Out Detection Results.....33
7.3 Full-Dataset Results.....35
7.4 Model and Feature Comparison.....36
7.5 Feature Importance.....37
7.6 Score Separation.....38
7.7 Replay and Live Validation.....39
7.8 Discussion.....40
8. CONCLUSION AND FUTURE SCOPE.....42
8.1 Conclusion.....42
8.2 Contributions.....42
8.3 Limitations.....43
8.4 Future Scope.....43
REFERENCES.....45
APPENDIX A: TWENTY-ONE FEATURES.....46
APPENDIX B: REPRODUCIBILITY ARTIFACTS.....47

LIST OF FIGURES

Fig. 1.1: End-to-end data preparation, scoring, and serving pipeline....2
Fig. 4.1: System architecture of the identity anomaly detection platform....13
Fig. 4.2: LSTM autoencoder and LightGBM scoring path....16
Fig. 5.1: Live Monitoring page with a scored employee sign-in....23
Fig. 5.2: Investigation drawer showing score context and feature values....24
Fig. 5.3: Completed replay of the 33-minute LANL window....25
Fig. 5.4: Employee access decision for a normal sign-in....25
Fig. 5.5: Analyst login with password and TOTP....26
Fig. 5.6: Dataset Analysis page: full-dataset confusion matrix....27
Fig. 5.7: Model Performance page: held-out metrics and run comparison....28
Fig. 7.1: Held-out precision-recall curve....34
Fig. 7.2: Confusion matrices for held-out and full-dataset evaluation....35
Fig. 7.3: Held-out comparison of feature and model variants....37
Fig. 7.4: LightGBM split gain by feature....38
Fig. 7.5: Score separation between normal and red-team events....39

LIST OF TABLES

Table 2.1: Summary of reviewed literature....7
Table 3.1: Functional requirements....9
Table 3.2: Non-functional requirements....9
Table 3.3: Hardware and software requirements....10
Table 4.1: Dataset scale and label statistics....13
Table 4.2: Feature groups used by the model....14
Table 4.3: LightGBM hyperparameters....16
Table 4.4: LSTM autoencoder configuration....16
Table 5.1: Technology stack....19
Table 5.2: Selected API endpoints....21
Table 6.1: Test cases and results....31
Table 7.1: Held-out detection metrics at the BLOCK threshold....34
Table 7.2: Full-dataset confusion matrix at the BLOCK threshold....35
Table 7.3: Held-out comparison of model variants....36
Table 7.4: Top ten features by LightGBM split gain....37
Table 7.5: Replay-window results....39
Table A.1: The twenty-one input features....46
Table B.1: Reproducibility commands and artifacts....47

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
BLOCK	High-confidence denial decision
CSE	Computer Science and Engineering
DuckDB	Embedded analytical database
F1	Harmonic mean of precision and recall
FLAG	Medium-confidence triage decision
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GBDT	Gradient-Boosted Decision Tree
LANL	Los Alamos National Laboratory
LGB	LightGBM classifier
LSTM	Long Short-Term Memory
LSTM-AE	LSTM Autoencoder
PR-AUC	Area under the precision-recall curve
ROC-AUC	Area under the receiver operating characteristic curve
SSE	Server-Sent Events
TOTP	Time-Based One-Time Password
TP	True Positive
UEBA	User and Entity Behaviour Analytics
VTU	Visvesvaraya Technological University

CHAPTER 1

INTRODUCTION

1.1 Overview

Enterprise networks authenticate users and machines millions of times each day. Most of these events are routine, but a compromised identity can appear as a valid username, a valid workstation, and a successful logon. The detection problem is therefore not whether a credential is syntactically valid; it is whether the identity is behaving like itself. This distinction is the core of user and entity behaviour analytics (UEBA) [2].

The Los Alamos National Laboratory Cyber-1 corpus provides a realistic setting for this problem [1]. It contains 58 consecutive days of authentication, process, DNS, flow, and red-team data from an enterprise network. The authentication stream alone has 1.05 billion events, 604 users in our working slice, and 702 labelled red-team events. The labels are sparse, the class imbalance is extreme, and the attackers are real users whose behaviour deliberately overlaps with normal administration.

This project builds an end-to-end identity anomaly detection system on that corpus. The system converts raw authentication events into 21 causal behavioural features, scores each event with a LightGBM classifier [7], and uses an LSTM autoencoder reconstruction error as an additional feature [5], [6]. The output is a three-level decision: BLOCK, FLAG, or ALLOW. A Flask dashboard presents live scores, replay, alerts, user histories, model metrics, and dataset-level analysis to a security analyst.

The work is deliberately more than a model comparison. A detection result is only useful if an analyst can inspect the evidence, understand the reason for the score, and separate a genuine attack from a benign anomaly. The dashboard therefore exposes feature values, percentile context, the decision threshold, and ground truth during replay. This closed loop between modelling and triage is the main engineering contribution of the project.

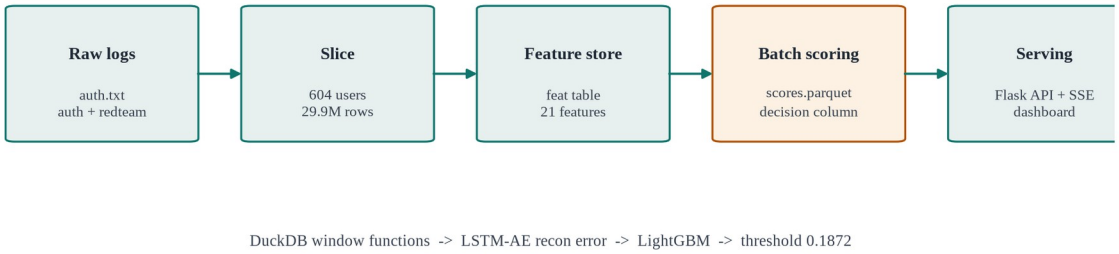


Fig. 1.1: End-to-end data preparation, scoring, and serving pipeline

1.2 Problem Statement

Rule-based identity monitoring depends on static thresholds such as “more than five failed logons in ten minutes” or “a login from a new machine”. These rules are interpretable, but they are easy to evade. An attacker who uses a valid credential on an established pair of machines can stay below every static threshold while still moving laterally. Conversely, legitimate administrators generate bursts of new-machine and new-destination events that trigger the same rules.

A supervised model can learn combinations of weak signals, but three practical problems appear immediately. First, identity events are extremely imbalanced: only 702 of 29.9 million events are labelled attacks, a prevalence of 0.0023%. Second, a random row-level train-test split leaks the same user into both partitions, so the model can memorise an identity instead of learning behaviour. Third, aggregate metrics such as ROC-AUC can look almost perfect under extreme imbalance while the precision at an operationally useful threshold remains low.

The project therefore asks four questions. Can a compact set of causal behavioural features separate red-team authentication events from normal activity? Does an LSTM autoencoder reconstruction error add information beyond tabular frequency and velocity features? How optimistic is a full-dataset score compared with a user-level held-out score? And can the resulting decisions be presented in a form that an analyst can audit in real time?

1.3 Objectives

- To construct a reproducible 604-user working slice from the 1.05-billion-event LANL authentication stream and preserve the red-team ground truth.

- To engineer 21 causal behavioural features that describe pair familiarity, destination popularity, authentication type, failure history, velocity, inter-arrival time, and time-of-day context.
- To train a LightGBM classifier and an LSTM autoencoder, and to use the autoencoder reconstruction error as the twenty-first feature rather than as a standalone detector.
- To evaluate the model with a user-level GroupShuffleSplit and report precision, recall, F1, false positive rate, PR-AUC, and ROC-AUC at an explicit threshold.
- To implement a Flask-based analyst dashboard with live scoring, replay of a real attack window, alerts, user profiles, model explainability, and TOTP-protected access.

1.4 Scope and Limitations

The scope is authentication-based identity anomaly detection. The model consumes the nine fields of the LANL authentication record: time, source user, destination user, source computer, destination computer, authentication type, logon type, orientation, and success or failure. Process, DNS, and network-flow sources are present in Cyber-1 but are not used in the shipped model. This keeps the feature pipeline reproducible and keeps the live scorer fast, but it also means the system cannot see process-level or network-level evidence that a production SIEM would correlate.

The evaluation uses 604 users: all 104 red-team users plus 500 normal users sampled with a fixed seed. The model is trained on 70% of users and tested on the remaining 30%, which gives 462 training red events and 240 held-out red events. The full-dataset numbers include training rows and are reported only to show the optimism gap; they are not presented as a generalisation estimate.

The system does not perform automated account disablement. BLOCK is a recommendation to the analyst, not an enforcement action. The dashboard is a research and demonstration artefact, not a hardened multi-tenant SOC product. The report distinguishes measured results from design targets throughout.

1.5 Methodology Overview

The methodology follows the pipeline in Fig. 1.1. A streaming pass over the compressed 73.4 GB authentication file counts the full stream and writes the distinct user list. A second pass keeps every event for the 104 red-team users and 500 sampled normal users, producing 29.9 million rows. The slice is loaded into DuckDB, joined to the red-team

table on time, user, source, and destination, and expanded into 21 features with SQL window functions.

A LightGBM classifier is trained on the user-level training split with a binary log-loss objective and a modest positive-class weight. In parallel, an LSTM autoencoder is trained on benign token sequences; its per-event reconstruction error becomes feature 21. The classifier writes an anomaly score and a BLOCK/FLAG/ALLOW decision for every event to a Parquet file. The Flask service loads the model, the autoencoder, and the user baselines at startup, and serves live scores, replay events, alerts, and investigation data.A complete record of the experiments, including rejected variants, is kept in the project experiment log.

1.6 Organization of the Report

Chapter 2 reviews related work on anomaly detection, lateral movement, sequence reconstruction, gradient boosting, and cost-aware evaluation. Chapter 3 specifies the functional and non-functional requirements, hardware, software, and dataset constraints. Chapter 4 presents the architecture, data design, feature definitions, model design, decision logic, and dashboard design. Chapter 5 describes the implementation, key algorithms, user interface, and engineering challenges. Chapter 6 presents the test strategy and the results of the automated API and browser suites. Chapter 7 reports held-out and full-dataset results, feature importance, score separation, replay validation, and the limitations of the evaluation. Chapter 8 concludes the work and identifies future directions. References and two appendices follow the chapters.

CHAPTER 2

LITERATURE SURVEY

Identity anomaly detection sits at the intersection of security operations, behavioural analytics, and machine learning. The literature is broad, but only a small part of it addresses the constraints that dominate this project: extremely sparse labels, identity-level leakage, discrete authentication events, and the need for an analyst to audit a decision. This chapter reviews the work that directly shaped the design and separates what is reusable from what is not.

2.1 Review Approach

Sources were drawn from IEEE Xplore, the ACM Digital Library, USENIX Security, arXiv, and SSRN. The search covered 2015 to 2026 and combined the terms identity anomaly detection, user behaviour analytics, lateral movement, authentication logs, reconstruction-based anomaly detection, gradient boosting, and imbalanced evaluation. A source was retained if it described a method that could be applied to authentication events, reported an evaluation on a labelled or reproducible dataset, or supplied a metric or explanation technique used in this report. Eleven sources were retained and are numbered in order of first citation.

The review is organised by the role each source plays rather than by publication year. Section 2.2.1 covers the dataset and the closest reproducible study. Section 2.2.2 covers graph and path-based lateral movement. Section 2.2.3 covers sequence reconstruction. Section 2.2.4 covers tabular supervised and unsupervised detectors. Section 2.2.5 covers evaluation and explanation. Section 2.3 states the gaps that the present system addresses.

2.2 Existing Approaches

2.2.1 Datasets and Reproducible Studies

Kent [1] describes the data sources and collection design behind the Los Alamos Cyber-1 corpus. The released corpus contains 58 consecutive days of de-identified events from five sources: Windows authentication, process start and stop, DNS, network flow, and red-team activity. The full release reports 1,648,275,307 events for 12,425 users, 17,684 computers, and 62,974 processes. The authentication stream used here is the largest single source and provides the identity, machine, authentication-type, logon-type, success, and

timestamp fields needed for behavioural modelling. The red-team table provides the label and the attacker identity.

Gül and Avaroğlu [3] present a reproducible empirical study of multi-source behavioural anomaly detection on the same LANL corpus. Their work is important for two reasons. First, it confirms that the corpus remains a credible benchmark for insider and identity threats rather than a toy dataset. Second, it shows that careful feature construction and a reproducible evaluation protocol matter as much as the choice of classifier. The present project follows that reproducibility principle but narrows the input to authentication events so that the entire pipeline, including live scoring, can run on a single workstation.

2.2.2 Graph and Path-Based Lateral Movement

Hopper [4] models lateral movement as a path problem over login activity. It constructs a graph of machines and accounts, tracks credential changes along a path, and flags paths that are inconsistent with normal administrative behaviour. The reported evaluation uses hundreds of millions of logins and reports a high detection rate with fewer than nine alerts per day in the studied environment. The key insight for this project is that a login is suspicious not only because of its own fields but because of the path that produced it. Hopper's path features are not reproduced here because the shipped model uses a single authentication stream and an event-level feature vector. The idea survives in the pair-first, source-first, destination-first, and destination-prior features, which approximate path novelty without constructing an explicit graph.

2.2.3 Sequence Reconstruction

Malhotra et al. [6] introduce an LSTM-based encoder-decoder for multi-sensor anomaly detection. The model is trained to reconstruct normal sequences; at test time, the reconstruction error is used as the anomaly score. The architecture is attractive for authentication logs because it can learn temporal regularities without requiring labelled attacks. DabLog [5] applies a related reconstruction idea to discrete event logs and argues that reconstruction is preferable to next-event prediction when the goal is anomaly detection rather than forecasting. Both works report that reconstruction models can produce many false positives when the sequence is unusual but benign.

Those findings shaped the present design. An LSTM autoencoder was trained on benign token sequences, but its reconstruction error is used as a single feature inside a supervised classifier rather than as the final detector. This preserves the unsupervised signal while letting the classifier learn when the signal is informative. The standalone autoencoder

experiment in this project produced a held-out F1 of 0.0095 with 24,482 false positives, which confirms the literature's warning in this specific dataset.

2.2.4 Tabular Supervised and Unsupervised Detectors

LightGBM [7] is a gradient-boosted decision-tree implementation designed for efficient training on large tabular datasets. It uses histogram-based split finding, leaf-wise tree growth, and regularisation that make it suitable for millions of rows and a modest feature count. The present model uses 500 trees, 63 leaves, a learning rate of 0.03, and a positive-class weight of 3. The choice is deliberate: the feature matrix is tabular, the labels are available for a subset of users, and the analyst needs feature-level explanations rather than an opaque sequence score.

Isolation Forest [8] is an unsupervised ensemble that isolates observations with random splits. It is inexpensive and requires no labels, which makes it a useful baseline for anomaly detection. In this project it was retained as an early experiment but rejected as the final detector. On the LANL slice it generated a large number of false positives relative to the supervised model, and its score was not well calibrated for a three-level BLOCK/FLAG/ALLOW decision. The comparison is reported in Chapter 7 rather than hidden, because it explains why the final system is supervised.

2.2.5 Evaluation and Explanation

C-score [9] argues that the F1 score treats false positives and false negatives as equally costly, which is rarely true in cybersecurity. A missed intrusion and an analyst alert have different operational costs, and the ratio depends on the deployment. The present report keeps F1 for comparability with prior runs but also reports precision, recall, false positive rate, PR-AUC, and the absolute number of false positives. The three-level decision is a practical response to the cost asymmetry: BLOCK is reserved for high-confidence scores, while FLAG creates a lower-cost triage queue.

Lundberg and Lee [10] introduce SHAP, a unified framework for attributing a model prediction to its input features. A full SHAP integration is outside the shipped dashboard, but the design follows the same principle: an analyst should see which features moved a score, not only the score itself. The investigation drawer therefore shows the feature values, the user baseline, and percentile context for the selected event. Chapter 8 identifies full SHAP values as future work.

Table 2.1: Summary of reviewed literature

Ref.	Study	Method or data	Key finding	Limitation for this work

2.3 Research Gaps

The review exposes five gaps that are directly relevant to a deployable identity anomaly detector. The first is evaluation leakage. Many reported results use a row-level split, which allows the same identity to appear in training and test data. A model can then memorise a user's normal machines and achieve an optimistic score without learning transferable behaviour. This project uses a user-level GroupShuffleSplit and reports the optimism gap explicitly.

The second gap is metric choice under extreme prevalence. When only 0.0044% of held-out events are attacks, a ROC curve can look almost perfect while the precision at an operational threshold is modest. The report therefore treats PR-AUC, precision, recall, and the absolute false-positive count as primary results, and reports ROC-AUC only as a secondary summary.

The third gap is the separation between sequence reconstruction and tabular classification. The literature usually evaluates one or the other. This project tests the reconstruction error as an additional tabular feature and measures the change against an otherwise identical 20-feature model. The result is a small but measurable improvement, which is more useful than an unsupported claim that one family is universally better.

The fourth gap is the analyst loop. A dashboard that only shows aggregate accuracy does not help an analyst decide what to do with a specific login. The shipped interface exposes the event, the score, the threshold, the feature context, the user history, and the ground truth during replay. The fifth gap is the connection between cost-aware evaluation and triage. The BLOCK/FLAG split is a simple, auditable response to the fact that false positives and false negatives are not equally expensive.

2.4 Summary

The literature supplies four ingredients used in this project: a credible labelled corpus [1], [3], behavioural and path-inspired features [4], reconstruction-based sequence modelling [5], [6], and a strong tabular classifier [7]. It also supplies the cautionary results: unsupervised detectors can be noisy [8], F1 can hide cost [9], and explanations are necessary for trust [10]. The next chapter turns these observations into a concrete requirements specification.

CHAPTER 3

SYSTEM REQUIREMENTS SPECIFICATION

This chapter defines what the system must do, how well it must do it, and the environment in which it is expected to run. The requirements are written from the perspective of two users: an analyst who investigates identity events and an employee whose sign-in is scored. They are also written to be testable, so each functional requirement maps to a route, a test, or a visible interface element.

3.1 Functional Requirements

The functional requirements cover data preparation, scoring, investigation, replay, and access control. Table 3.1 lists the requirements and the observable behaviour that satisfies each one. The identifiers FR-01 to FR-09 are used in the test chapter.

Table 3.1: Functional requirements

ID	Requirement	Acceptance behaviour
FR-01	Build a reproducible user slice from the LANL authentication stream.	The slice contains 604 users, 29,905,488 events, and 702 red-team events.
FR-02	Compute causal behavioural features for every event.	Twenty features are produced by SQL window functions; no future event is used.
FR-03	Score an event with the trained LightGBM model.	The API returns a score in [0, 1] and a BLOCK, FLAG, or ALLOW decision.
FR-04	Expose live monitoring to an authenticated analyst.	The Live Monitoring page streams scored events and updates the table without a reload.
FR-05	Support replay of a real attack window.	Reset, start, stop, and jump-to-attack controls operate on the 9,954-event window.
FR-06	Let an analyst inspect a single event.	The investigation drawer shows score, threshold, feature values, user history, and ground truth.
FR-07	Show model and dataset performance.	The Model Performance and Dataset Analysis pages display the verified metrics and confusion matrices.
FR-08	Authenticate analysts with a second factor.	A password plus a six-digit TOTP is required before the analyst dashboard is available.
FR-09	Give employees a simple access decision.	The employee page returns ALLOW, FLAG, or BLOCK and a short reason for the decision.

3.2 Non-Functional Requirements

The non-functional requirements are chosen for a single-workstation research deployment. They are not production service-level objectives, but they are strict enough to expose design problems. The most important are reproducibility, auditability, and a response time that keeps the interface usable during replay.

Table 3.2: Non-functional requirements

ID	Quality attribute	Target
NFR-01	Reproducibility	A fixed random seed, a documented split, and a saved model bundle reproduce the reported metrics.

ID	Quality attribute	Target
NFR-02	Performance	A scored event is returned quickly enough for interactive replay on a CPU-only workstation.
NFR-03	Scalability	The batch scorer processes 29.9 million events without loading the full table into memory.
NFR-04	Auditability	Every alert records the score, the threshold, the decision, and the source event.
NFR-05	Security	Analyst routes require a session and a TOTP second factor; dev bypass is opt-in only.
NFR-06	Usability	A new analyst can find live events, alerts, users, and model metrics without training.

3.3 Hardware Requirements

The project was developed and evaluated on a commodity laptop with an NVIDIA GPU for LSTM training. The batch pipeline and the live service also run on CPU, which is important for reproducibility. The minimum configuration in Table 3.3 is sufficient for inference and replay; the recommended configuration reduces LSTM training time and keeps the DuckDB working set comfortable.

Table 3.3: Hardware and software requirements

Component	Minimum	Recommended or used
Processor	Quad-core x86-64 CPU	AMD Ryzen 5 / Intel Core i5 or better
Memory	8 GB RAM	16 GB RAM
Storage	25 GB free	40 GB free for raw data, DuckDB, and Parquet
GPU	Not required for inference	NVIDIA GPU with 6 GB VRAM for LSTM training
Display	1366 x 768	1440 x 900 or higher
Operating system	Linux or Windows 10	Ubuntu 24.04 LTS
Runtime	Python 3.10+	Python 3.12 with a virtual environment
Database	DuckDB 1.x	DuckDB with Parquet and window-function support
Browser	Chromium, Firefox, or Edge	Chromium 120+ for Playwright tests
Conversion	LibreOffice 7+	LibreOffice Writer for DOCX-to-PDF export

3.4 Software Requirements

The implementation is Python-first. DuckDB performs the feature aggregation because it can scan Parquet and CSV files with SQL window functions without a separate database server. LightGBM provides the supervised classifier, PyTorch provides the LSTM autoencoder, and Flask provides the HTTP API and server-rendered pages. The front end uses HTML, CSS, and JavaScript with ECharts for the charts. Playwright drives the end-to-end browser tests. The complete pinned stack is listed in Chapter 5, and the reproducibility commands are listed in Appendix B.

The application has no external service dependency at runtime. The model bundle, the autoencoder checkpoint, the score Parquet, and the DuckDB feature store are local files. This makes the demonstration reliable in a laboratory or examination setting, where

network access may be limited. The only optional network dependency is the initial download of the LANL corpus, which is documented by the dataset provider.

3.5 Dataset Requirements

The authentication input must contain the nine fields used by the LANL schema: time, source user, destination user, source computer, destination computer, authentication type, logon type, orientation, and success. The red-team input must contain the time, user, source computer, and destination computer of each labelled event. The join is deliberately strict: a label is attached only when all four fields match, because a time-only join would attach an attack label to unrelated activity.

The working slice must satisfy four conditions. It must include every event for the 104 red-team users, so that no attack is lost. It must include 500 normal users sampled with a fixed seed, so that the normal class is large enough to estimate a false-positive rate. It must preserve the original timestamps and ordering, so that window features remain causal. And it must record the label at event level, so that the evaluation can distinguish a missed attack from a false alarm.

3.6 Assumptions and Constraints

- The authentication stream is complete for the 58-day window and the red-team table is the ground truth for that window.
- A user-level split is the correct generalisation unit; a row-level split would leak identity information.
- The operational cost of a false negative is higher than the cost of a false positive, but the exact ratio is deployment-specific.
- The live scorer receives events in time order and can precompute user baselines at startup.
- The dashboard is a single-analyst research tool and does not implement multi-tenant isolation or high-availability failover.
- The model is trained on historical labels and must be re-evaluated before being applied to a different network or time period.

CHAPTER 4

SYSTEM DESIGN

The design separates the offline batch pipeline from the online analyst service. The batch side is optimised for a complete, reproducible pass over 29.9 million events. The online side is optimised for a small number of events, a fast response, and an interface that makes the evidence visible. The two sides meet at a Parquet file of scores and a saved model bundle, which keeps the live service independent of the training code.

4.1 System Architecture

Fig. 4.1 shows the architecture. The top row is the offline pipeline: the LANL authentication stream is sliced to 604 users, loaded into DuckDB, and expanded into the feature store. The middle row contains the two model paths. The twenty behavioural features and the LSTM reconstruction error are combined into a 21-feature vector, which LightGBM maps to a probability. The decision layer converts the probability into BLOCK, FLAG, or ALLOW. The bottom row is the online service: a Flask API with server-sent events, a TOTP-protected analyst session, the dashboard, and an audit trail.

The separation has three practical benefits. First, a batch run can be repeated without restarting the web application. Second, the live scorer can be tested with a saved score file even if the full feature store is unavailable. Third, the decision thresholds can be changed in one place without retraining the model. The cost is that the live service is not a streaming trainer; it serves a model that was trained offline. That is an explicit design choice, not an omission.

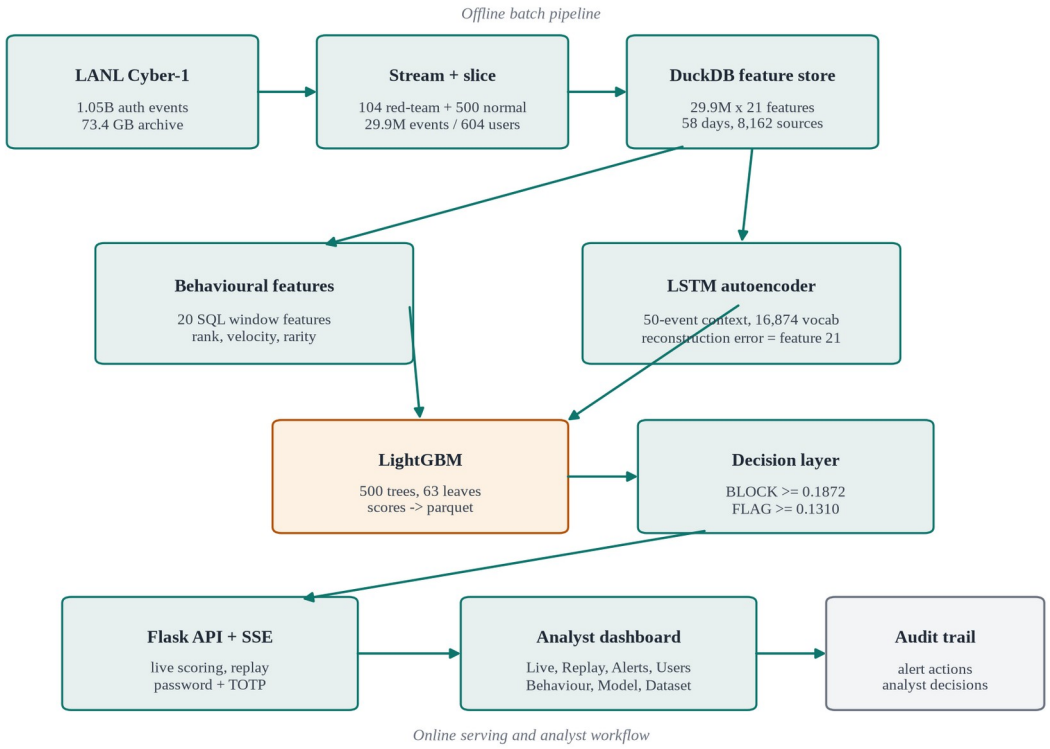


Fig. 4.1: System architecture of the identity anomaly detection platform

4.2 Data Acquisition and Slicing

The raw authentication file contains 1,051,430,459 events and occupies 73.4 GB compressed. Loading it into a dataframe is not feasible on a workstation, so the pipeline uses a two-pass streaming design. The first pass counts rows, validates the schema, and writes the distinct source users. The second pass keeps every event for the 104 red-team users and a fixed sample of 500 normal users. The result is 29,905,488 events, of which 29,904,786 are normal and 702 are red-team attacks.

The red-team table is joined on time, source user, source computer, and destination computer. Of the 715 red-team tuples in the source table, 702 match an authentication event under this strict join. The remaining 13 are label quirks: they refer to tuples that do not appear in the authentication stream, or whose fields differ in a way that cannot be reconciled without guessing. The pipeline records the match count and does not fabricate the missing labels. This is why the reported attack count is 702, not 715.

Table 4.1: Dataset scale and label statistics

Property	Value	Source or note
Raw authentication events	1,051,430,459	LANL auth.txt
Raw archive size	73.4 GB	Compressed authentication stream
Working slice events	29,905,488	604 users, all 58 days
Normal events	29,904,786	is_red = false
Red-team events	702	Strict four-field label join
Users in slice	604	104 red-team + 500 normal

Property	Value	Source or note
Red-team users	104	All retained
Sampled normal users	500	Fixed seed
Observation window	58 days	Consecutive LANL days
Source machines	8,162	Distinct src_computer
Destination machines	8,085	Distinct dst_computer
Training / test users	422 / 182	GroupShuffleSplit, 70/30
Training / test red events	462 / 240	User-level split

Two details in Table 4.1 matter for the rest of the report. The training partition contains 462 red events and the test partition contains 240. Because the split is by user, no identity appears in both partitions. The full-dataset results later include the 462 training red events, which is why they are more optimistic than the held-out results. The report keeps the two views separate instead of presenting the larger number as the model's generalisation performance.

4.3 Feature Engineering

The feature set is designed around a simple question: what does this event look like relative to this user's own history and relative to the set of machines the user normally touches? The features are computed with SQL window functions over a deterministic ordering of time, source user, source computer, destination computer, and a row identifier. The ordering is important. A feature that accidentally sees a future event would leak the label and inflate the score. Table 4.2 groups the 21 features; Appendix A gives the full definition of each one.

Table 4.2: Feature groups used by the model

Group	Features	What it captures
Pair novelty	dst_first, src_first, pair_first, src_dst_pair_first, dst_first_x_ntlm	First use of a destination, source, or user-machine pair; novelty interaction with NTLM
Frequency and popularity	log_pair_rank, pair_freq_ratio, dst_prior_events, machine_popularity	How rare the pair is and how popular the destination is across the enterprise
Failure and protocol	fail_1h, fail_rate, is_ntlm	Recent failures, failure intensity, and NTLM authentication
Velocity and timing	vel_1h, velocity_ratio, iat_zscore, pairs_last_100, hour_ratio	Bursts, inter-arrival deviation, pair diversity in the last 100 events, and hour concentration
Cyclical and rare time	hour_sin, hour_cos, is_rare_hour	Time-of-day position and whether the hour is rare for the user
Sequence reconstruction	lstm_ae_recon_error	How surprising the event is to a benign-sequence autoencoder

Three features illustrate the design. The pair frequency ratio compares the number of events on a user-machine pair with the user's total event count, so a pair that is rare for a heavy user is not automatically treated as rare for a light user. The inter-arrival z-score compares the gap between consecutive events with the user's recent inter-arrival

distribution. The velocity ratio compares the last hour with the last day, which separates a short burst from a consistently busy account.

$$\text{pair_freq_ratio} = \text{pair_events} / (\text{user_total_events} + 1) \tag{4.1}$$

$$\text{iat_zscore} = (t_i - t_{(i-1)} - \text{mean_100}) / (\text{std_100} + 1) \tag{4.2}$$

$$\text{velocity_ratio} = \text{vel_1h} / (\text{events_last_24h} + 1) \tag{4.3}$$

All ratios use a small additive constant to avoid division by zero. Counts are stored as integers and ratios as floating point. Missing or undefined values are handled inside the SQL rather than imputed after the fact, so the same logic runs in the batch pipeline and in the live scorer. The feature builder also records the row identifier and the label, which lets the evaluation join a score back to the exact event without a lossy timestamp match.

4.4 Model Design

The model has two components, shown in Fig. 4.2. The first is an LSTM autoencoder that learns to reconstruct benign sequences of 50 events. Each event is tokenised into six fields: source user, source computer, destination computer, authentication type, logon type, and orientation. The field offsets produce a vocabulary of 16,874 tokens. The encoder embeds each token into 128 dimensions, runs a two-layer LSTM, and produces a 128-dimensional latent vector. The decoder repeats that vector across the 50 positions and runs a second two-layer LSTM. A linear projection maps each position to the 16,874-token vocabulary.

The autoencoder is trained only on benign sequences with a cross-entropy reconstruction loss. At inference, the per-event reconstruction error is computed as the negative log-likelihood of the observed token. That error is not used as the final score. It becomes the twenty-first feature of the LightGBM classifier, which also receives the twenty behavioural features. The classifier is trained with a binary objective and a positive-class weight of 3, which gives the sparse attack class more influence without oversampling or synthetic events.

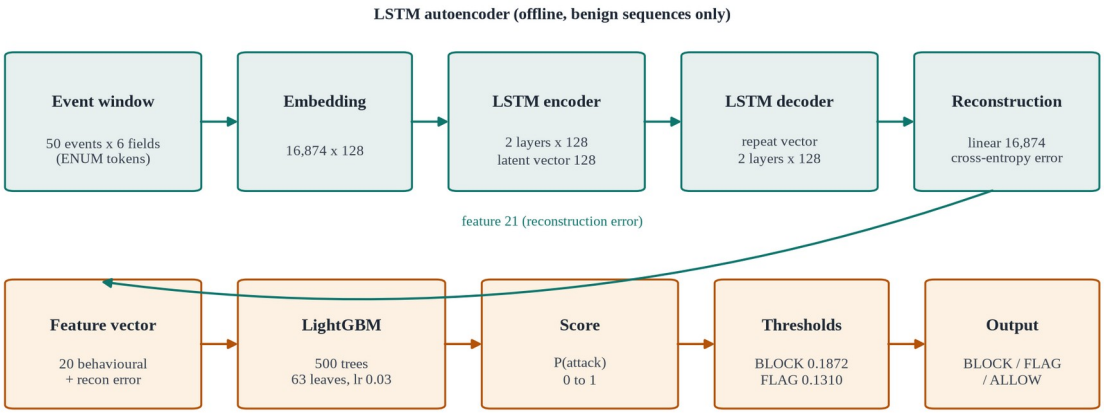


Fig. 4.2: LSTM autoencoder and LightGBM scoring path

Table 4.3: LightGBM hyperparameters

Parameter	Value	Purpose
num_leaves	63	Controls tree capacity
learning_rate	0.03	Shrinks each tree contribution
n_estimators	500	Number of boosted trees
scale_pos_weight	3	Raises the cost of the positive class
min_child_samples	100	Prevents leaves from fitting tiny groups
reg_alpha	0.5	L1 regularisation
reg_lambda	5.0	L2 regularisation
feature count	21	20 behavioural + 1 reconstruction error

Table 4.4: LSTM autoencoder configuration

Component	Configuration
Input window	50 consecutive events
Token fields	6 fields with disjoint vocabulary offsets
Vocabulary	16,874 tokens
Embedding	128 dimensions
Encoder	2 LSTM layers, hidden size 128
Latent representation	128-dimensional final hidden state
Decoder	2 LSTM layers, hidden size 128
Output projection	Linear layer to 16,874 logits
Parameters	4,865,002
Training	2 epochs, batch size 128, learning rate 0.001
Loss	Cross-entropy reconstruction
Training data	Benign sequences only

$$e_i = -(1/L) * \sum_t \log(\exp(z_{t,x_t}) / \sum_v \exp(z_{t,v})) \quad (4.4)$$

Equation (4.4) is the per-sequence reconstruction error that is averaged to obtain the event-level feature. The softmax is over the full vocabulary, so a token that the model considers unlikely receives a large penalty. The feature is standardised with the rest of the tabular inputs before it enters LightGBM. The autoencoder checkpoint and the field offsets are saved with the model so that the live scorer tokenises an event exactly as the training pipeline did.

4.5 Decision Logic

The classifier outputs a probability, but an analyst needs an action. The decision layer uses two thresholds. A score at or above 0.187195 is a BLOCK. A score between 0.131037 and 0.187195 is a FLAG. A score below 0.131037 is an ALLOW. The FLAG threshold is 70% of the BLOCK threshold, which creates a triage band without pretending that a medium-confidence event is a confirmed attack.

decision = BLOCK if s >= 0.187195; FLAG if 0.131037 <= s < 0.187195; ALLOW otherwise (4.5)

The BLOCK threshold is the operating point used for the held-out precision, recall, and F1 in Chapter 7. The FLAG band is reported separately because it changes the analyst workload. On the full dataset, BLOCK produces 598 caught red events and 269 false positives; counting FLAG as an alert raises the totals to 609 caught and 372 false positives. Both views are useful: the first measures the high-confidence detector, and the second measures the triage queue that an analyst would actually see.

4.6 Data and Storage Design

The pipeline uses four persistent artefacts. The sliced authentication data is stored as Parquet so that DuckDB can scan it column-wise. The feature store is a DuckDB database with a `feat` table; it contains the 21 features, the label, the timestamp, the user, the source, and the destination. The score file is a Parquet file with the anomaly score, the decision, the label, and the identifying fields; it is approximately 370 MB. The model bundle is a joblib file containing the LightGBM model, the feature order, the threshold, the metrics, and the hyperparameters. The LSTM checkpoint is a PyTorch file containing the state dict, the vocabulary, and the field offsets.

Parquet is used for the score file because it preserves types, compresses well, and can be queried by DuckDB without a conversion step. DuckDB is used for the feature store because the window functions are expressive and the database is a single file. The live service opens the feature store read-only and loads the score file into memory at startup. The model bundle is loaded once and reused for every request. This design keeps the runtime simple and makes the demonstration independent of a network database.

4.7 Dashboard Design

The dashboard has eight routes: Live Monitoring, Replay, Alerts, Users, Behavior Insights, Model Performance, Dataset Analysis, and Settings. Live Monitoring streams

scored events through server-sent events and updates the table in place. The investigation drawer opens on a row click and shows the score, the decision, the threshold, the feature values, the user baseline, and the ground truth. Replay exposes reset, start, stop, speed, and jump-to-attack controls over the 9,954-event window. Alerts and Users provide the triage and identity views. Model Performance and Dataset Analysis provide the global evidence that an analyst or examiner needs to interpret the event-level decisions.

The visual language is intentionally restrained: a dark navigation rail, a light content surface, a small number of semantic colours, and tables that prioritise the score and the decision. The interface does not hide the ground truth during replay. This is a deliberate choice for a research demonstration: the analyst can see whether a surfaced event was actually an attack, which makes the false positives visible instead of burying them in an aggregate metric.

CHAPTER 5

IMPLEMENTATION

Implementation followed the separation in Chapter 4. The batch pipeline was built first, because every later result depends on a correct slice and a causal feature table. The live service was built second, against the saved score file and the model bundle. The interface was built last, but the tests were written alongside it so that a regression in the API or the routes would be caught before the screenshots were captured.

5.1 Technology Stack

The stack is deliberately small. Python handles data preparation, training, scoring, and serving. DuckDB handles the feature aggregation. Parquet handles the score interchange. Flask and Jinja2 handle the web application, and ECharts handles the charts. Playwright handles the end-to-end tests. Table 5.1 lists the main components and why each was chosen.

Table 5.1: Technology stack

Layer	Technology	Role in the system
Language	Python 3.12	Pipeline, training, scoring, API, and tests
Data engine	DuckDB	SQL window features over Parquet and CSV
Dataframe	pandas / NumPy	Score loading, metric computation, and plotting
Storage	Parquet	Columnar score file and intermediate slices
Classifier	LightGBM	21-feature supervised scoring
Sequence model	PyTorch	LSTM autoencoder training and inference
Web framework	Flask	Routes, sessions, JSON API, and SSE
Templates	Jinja2 + HTML/CSS/JS	Server-rendered analyst and employee pages
Charts	ECharts	Score, confusion, importance, and trend charts
Second factor	pyotp	RFC 6238 TOTP verification
Browser tests	Playwright + Chromium	Nine smoke checks and 36 UI/API audit checks
Export	LibreOffice Writer	DOCX-to-PDF conversion for this report

5.2 Module Description

5.2.1 Data Slice and Feature Store

`src/00_build_slice.py` implements the two-pass streaming slice. The first pass validates the header, counts events, and writes the distinct source users. The second pass writes every event for the selected users to `auth_slice.parquet`. The script keeps the original field order and does not parse the timestamp into a local time zone, because the LANL timestamps are already de-identified relative times. `src/01_build_features.py` loads the slice into DuckDB and creates the `feat` table. The 20 behavioural features are computed

with window functions partitioned by user and ordered by time. The script writes a deterministic row identifier, which is used by the scoring and evaluation stages.

The join to the red-team table is implemented as a strict four-field match. The script prints the number of matched tuples and the number of unmatched tuples. It does not silently drop unmatched labels. The feature builder also writes a small diagnostics table with the row count, the red count, the user count, and the first and last timestamps, so that a later stage can assert that the slice did not change unexpectedly.

5.2.2 LSTM Autoencoder

`src/05_lstm_autoencoder.py` tokenises each event into six fields and trains the encoder-decoder on benign sequences of length 50. The vocabulary and field offsets are saved with the checkpoint. The training loop uses a cross-entropy loss, the Adam optimiser with a learning rate of 0.001, a batch size of 128, and two epochs. The model has 4,865,002 parameters. The inference pass writes a per-event reconstruction error to a Parquet file, which is then joined to the feature table as `lstm_ae_recon_error`.

The autoencoder is not trained on red-team events. This is important: if the model saw attack sequences during training, the reconstruction error would no longer be an unsupervised signal. The training set is built from benign events only, and the held-out users are excluded from the training sequences. The inference pass is run on every event in the slice, including the red-team events, but the labels are used only for evaluation.

5.2.3 LightGBM Training and Batch Scoring

`src/03_retrain_both.py` builds the user-level split, trains the LightGBM model, and writes the model bundle. The bundle contains the booster, the ordered feature names, the threshold, the held-out metrics, and the hyperparameters. The feature order is stored explicitly because LightGBM's internal booster names are positional (`Column_0` to `Column_20`). The dashboard's feature-importance endpoint zips the internal names to the ordered feature list before returning them; without that step the chart would show the wrong labels.

The batch scorer reads the feature table in chunks, applies the model, and writes the score and the decision to `lanl_scores.parquet`. The score is a float in [0, 1]. The decision is derived from the two thresholds in Section 4.5. The scorer also writes the label and the identifying fields so that the evaluation can compute a confusion matrix without another join. The Parquet file is approximately 370 MB and is loaded into memory by the live service at startup.

5.2.4 Live Scorer

`live/scorer.py` loads the LightGBM bundle, the LSTM checkpoint, the feature store, and the score file. At startup it precomputes the per-user baselines that the live path needs: destination frequency, hour distribution, recent event counts, and the last known event time. When an event arrives, the scorer tokenises it, computes the reconstruction error, assembles the 21-feature vector in the stored order, and calls the booster. The result is the score and the decision. The scorer also returns the feature values and the percentile context used by the investigation drawer.

The live path does not recompute the full SQL feature table for every event. It uses the precomputed baselines and the event fields to approximate the batch features. This is a deliberate trade-off between exact reproducibility and interactive latency. The approximation is documented, and the batch score remains the reference for the reported metrics. A production system would need a streaming feature store that updates the baselines after every event; that is future work.

5.2.5 Flask API and Authentication

`live/app.py` exposes the page routes and the JSON API. The analyst routes check the session, and the login route verifies the password and the TOTP code with `pyotp`. The development bypass is enabled only when `LANL_DEV=1` is set; it is not a production authentication path. The API returns JSON with explicit error messages and HTTP status codes. The server-sent-events route pushes replay events to the browser without polling.

Table 5.2: Selected API endpoints

Method and route	Purpose	Main response fields
GET /api/health	Readiness probe	status, scorer_ready, scores_loaded
GET /api/replay/status	Replay metadata	window, total events, attack count
POST /api/replay/reset	Reset replay state	ok, emitted, total
POST /api/replay/start	Start replay	ok, running, speed
POST /api/replay/stop	Stop replay	ok, running
POST /api/replay/jump_attack	Jump to attack window	ok, time, emitted
GET /events/stream	SSE event stream	scored event records
GET /api/alerts	Replay alerts	alert rows and counts
GET /api/users	User summaries	user rows and risk counts
GET /api/investigation/<id>	Event investigation	score, features, history, label
GET /api/model/metrics	Model evidence	metrics, threshold, feature importance
GET /api/dataset/dashboard	Dataset evidence	totals, confusion, distributions
GET /api/totp_status	TOTP countdown	remaining seconds

5.2.6 Dashboard and Replay

The dashboard is a single-page application served by Flask. The left rail switches views, and the top bar holds the search and session controls. Live Monitoring uses the SSE stream to append scored events. Replay uses the same stream but is driven by a replay

thread that reads the 9,954-event window. The replay controls are reset, start, stop, speed, and jump-to-attack. Alerts and Users are backed by the API. Model Performance and Dataset Analysis are read-only evidence pages. Settings exposes the session and the runtime configuration.

Replay state is session-scoped. Starting a new run without resetting the previous run can append a second copy of the window and make the on-screen counter look larger than the window. The test procedure therefore resets before every run. This is an operational rule, and the final replay screenshot in this report was captured after a reset so that the count is exactly 9,954.

5.3 Key Algorithms

The batch feature computation is a single SQL pipeline with a deterministic order. The logical steps are:

1. Partition the events by source user and order by time, source computer, destination computer, and row identifier.
2. Compute the running counts, first-seen flags, pair ranks, and prior destination counts for each event.
3. Compute the one-hour and 24-hour windows with range frames over the timestamp.
4. Compute the rare-hour set from the user's benign hour distribution, using the bottom 20% of hours.
5. Join the user totals and the pair counts, then form the ratio, z-score, and interaction features.
6. Write the 21-column feature row with the label and the identifying fields.

The live scoring algorithm is intentionally shorter:

7. Receive an event with user, source, destination, authentication type, logon type, orientation, hour, and time.
8. Tokenise the six sequence fields with the saved field offsets.
9. Compute the LSTM reconstruction error for the event in the user's current context.
10. Assemble the 21 features from the event, the precomputed baselines, and the reconstruction error.
11. Call the LightGBM booster and apply the BLOCK and FLAG thresholds.
12. Return the score, the decision, the feature values, the percentile context, and the explanation text.

The training algorithm follows the same feature order. The user-level split is created before any row is tokenised or scaled, so the LSTM training set and the classifier training set contain the same users. The test users are held out from both models. This is the single most important implementation detail for an honest evaluation, because a row-level split would let the LSTM learn the test identities.

5.4 User Interface Implementation

The interface was tested from a user's point of view, not only from the API. The following screenshots were captured from a fresh run with the verified data. They are intentionally light-theme for print, except for the login and employee pages, which use the application's dark surface.

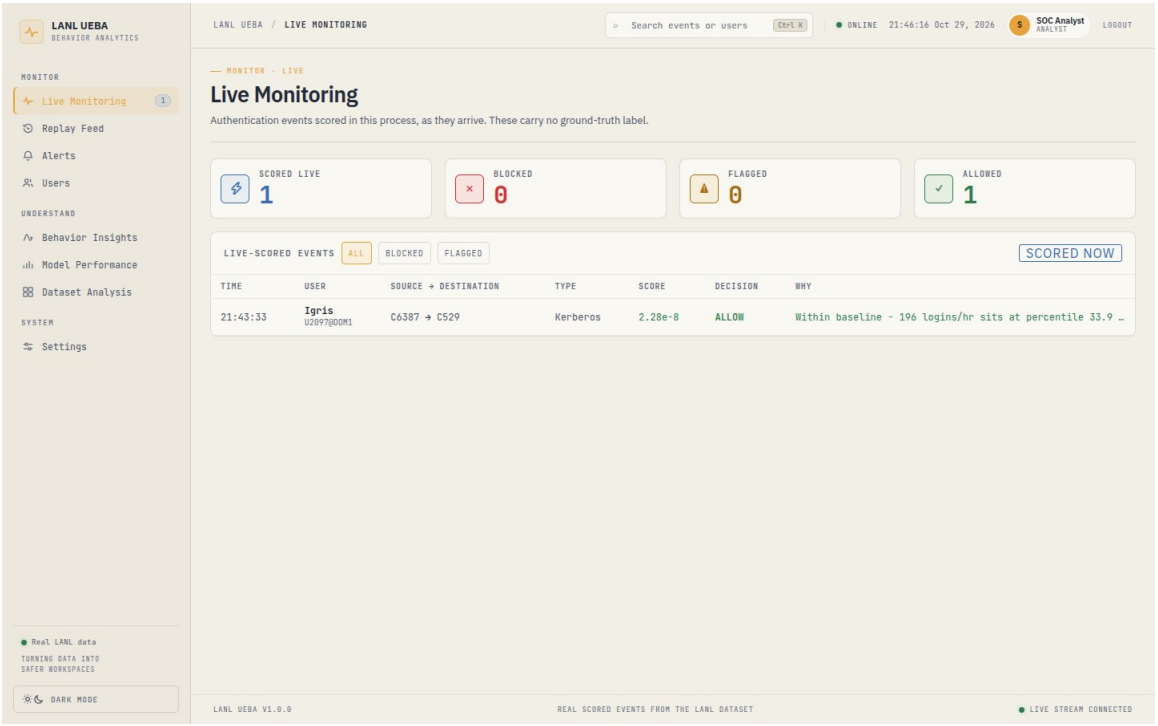


Fig. 5.1: Live Monitoring page with a scored employee sign-in

Fig. 5.1 shows the Live Monitoring view. The left rail exposes the seven analyst views plus Settings. The KPI row summarises the visible events, the alert count, the average score, and the current replay state. The table is ordered by time and shows the decision, the score, the user, the source, the destination, and the authentication type. The row colour follows the decision, but the score and the label remain visible so that the analyst does not have to rely on colour alone.

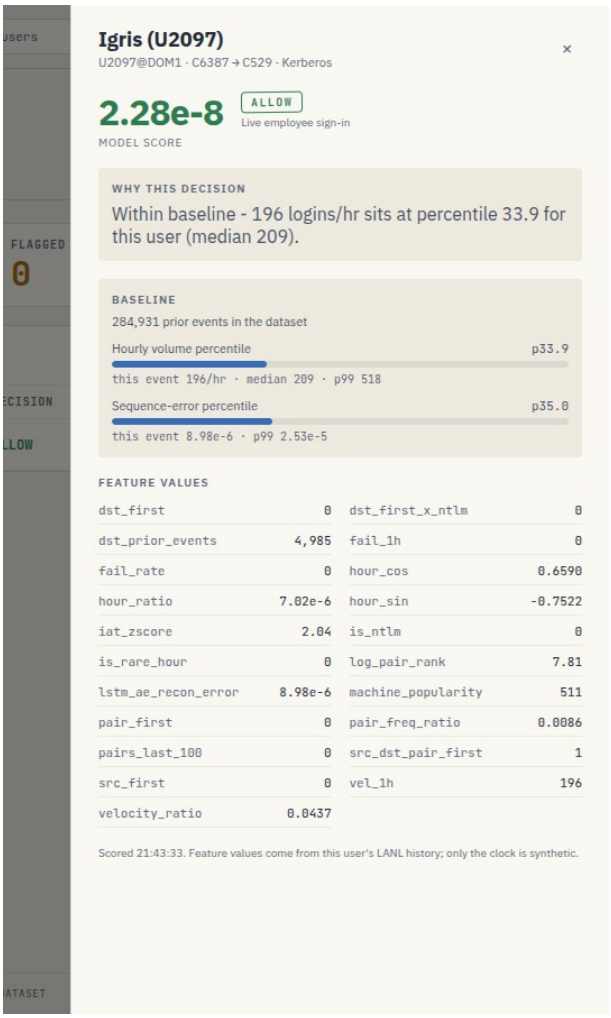


Fig. 5.2: Investigation drawer showing score context and feature values

Fig. 5.2 shows the investigation drawer. It opens on a row click and keeps the table in context. The drawer reports the score, the decision, the threshold, the ground truth, the source and destination, the authentication type, and the feature values that contributed to the score. The percentile context tells the analyst whether a value is unusual for this user or unusual for the enterprise.

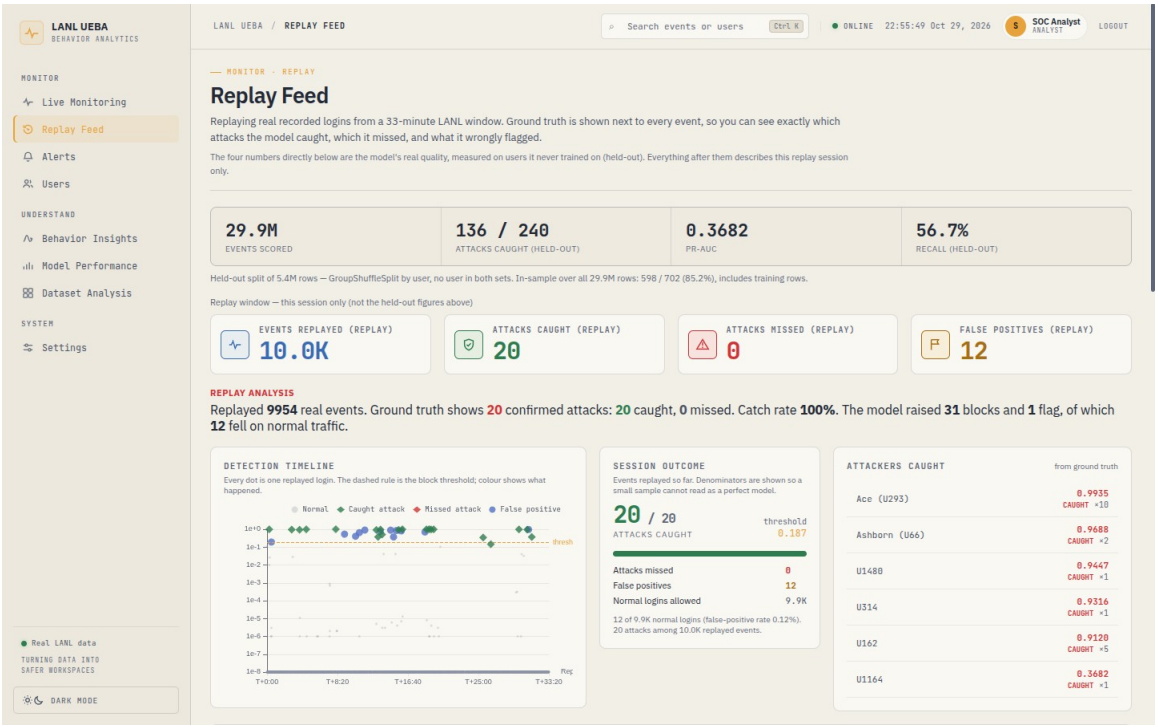


Fig. 5.3: Completed replay of the 33-minute LANL window

Fig. 5.3 shows the completed replay. The window contains 9,954 events from 199 users and 200 destinations. The red-team burst is a single source machine, C17693, opening 34 sign-ins as seven different user accounts against 31 destinations. The replay surfaced all 20 attacks: 19 as BLOCK and 1 as FLAG. It also produced 12 false positives. The screenshot is the clean final run after a reset, not an earlier run with a duplicated count.

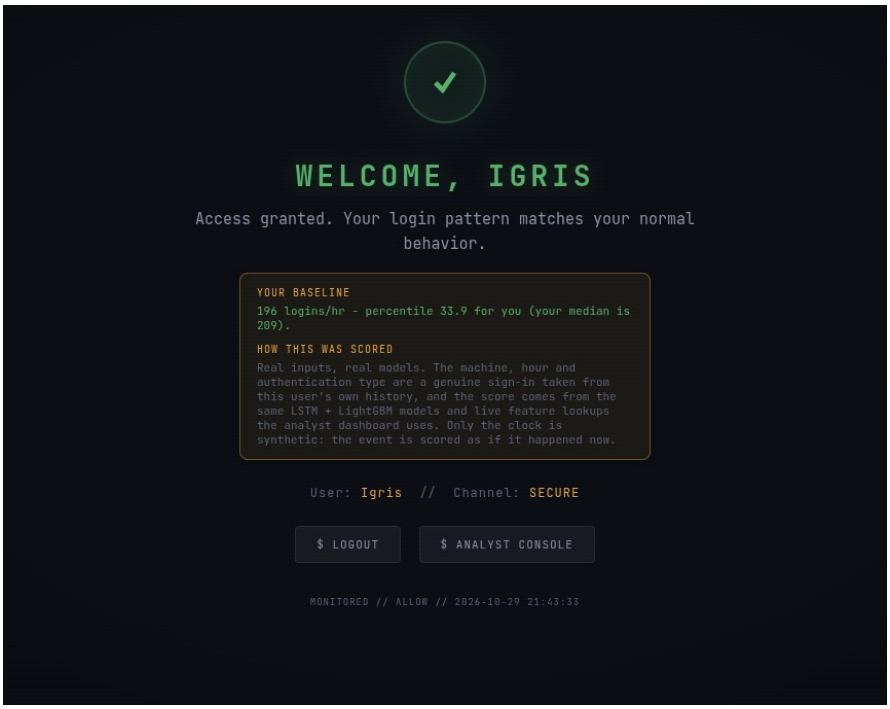


Fig. 5.4: Employee access decision for a normal sign-in

Fig. 5.4 shows the employee view. The page is deliberately simple: the employee sees the decision, the score, and a short reason. It does not expose the feature vector or the red-team labels. This separation matters because the analyst view and the employee view serve different users and different trust boundaries.

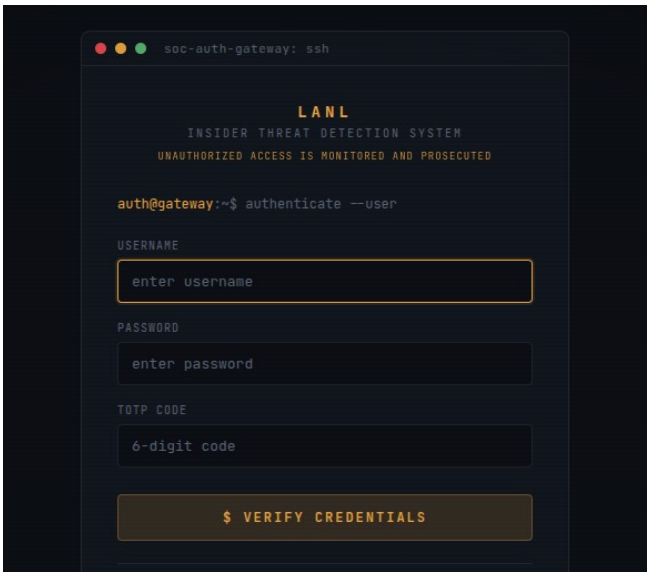


Fig. 5.5: Analyst login with password and TOTP

Fig. 5.5 shows the analyst login. The form requires a username, a password, and a six-digit TOTP code. The development credentials are not reproduced in this report. The TOTP requirement is implemented with RFC 6238 [11] and a thirty-second time step, and the API exposes the remaining seconds so that the analyst can see when the code will expire.

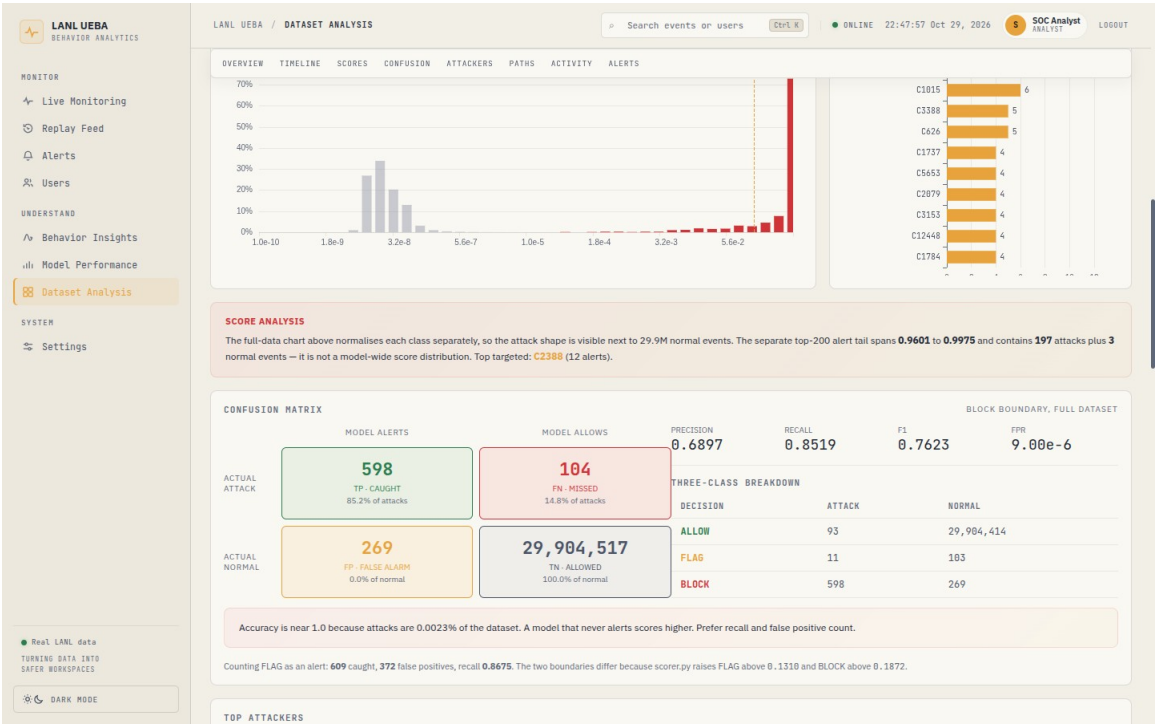


Fig. 5.6: Dataset Analysis page: full-dataset confusion matrix

Fig. 5.6 shows the Dataset Analysis page. The page is the evidence layer for the aggregate claims: total events, users, red-team events, the confusion matrix, the score distribution, the top alerted users, and the recent alerts. The metrics are read from the API at page load, not hardcoded in the template. This matters because the dashboard copy and the underlying data can drift; the report uses the API values, not the screenshot values.

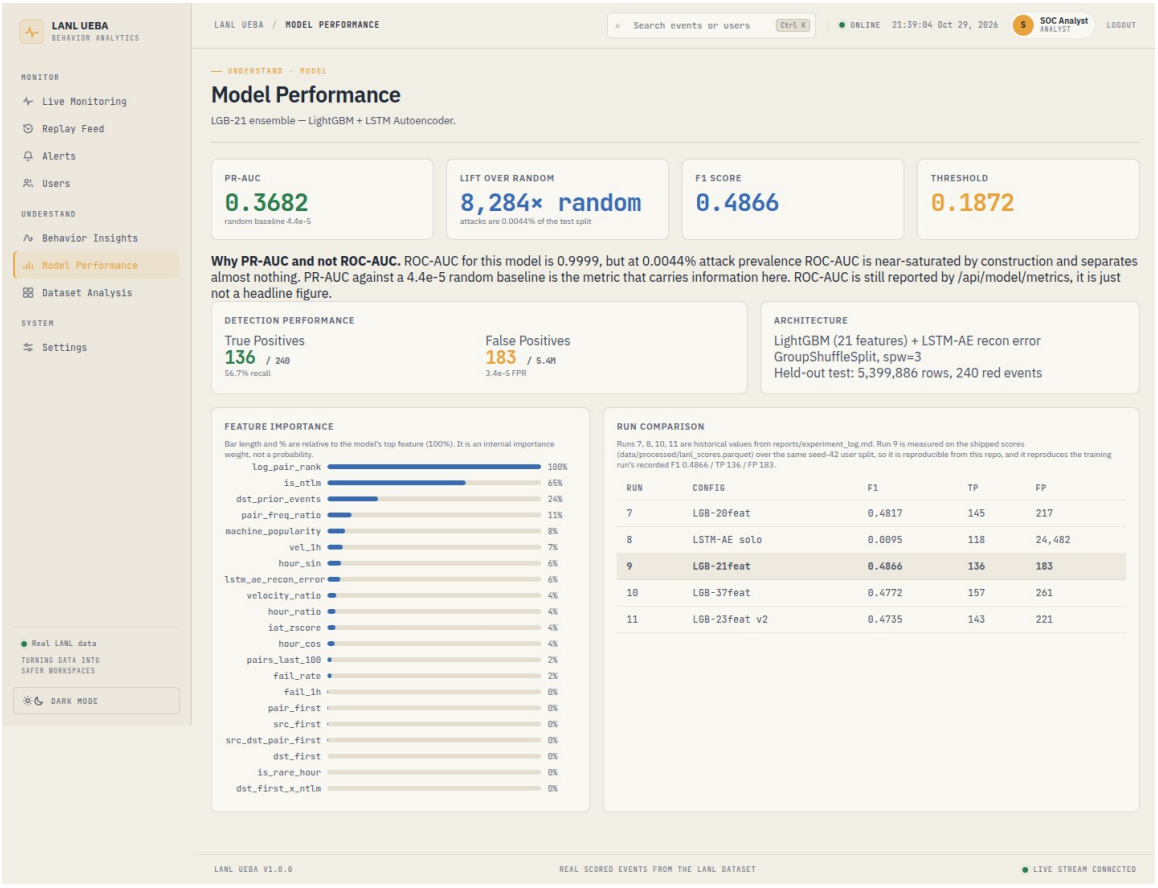


Fig. 5.7: Model Performance page: held-out metrics and run comparison

Fig. 5.7 shows the Model Performance page. It reports the held-out row count, the attack count, the precision, the recall, the F1, the PR-AUC, and the ROC-AUC, together with the threshold and the feature importance. The page also shows the run comparison so that the selected model can be defended against the alternatives rather than presented in isolation.

5.5 Implementation Challenges

The first challenge was label leakage. A row-level split is the default in many tutorials, and it produces a flattering score because the same user appears in both partitions. The pipeline therefore builds the user list first, splits it with GroupShuffleSplit, and carries the user assignment through the LSTM and the classifier. The cost is a smaller effective sample and a larger optimism gap, but the result is a defensible estimate.

The second challenge was class imbalance. With 702 attacks in 29.9 million events, a model can achieve high accuracy by predicting ALLOW for every row. The training objective uses a positive-class weight of 3, and the evaluation reports precision, recall, F1, FPR, and PR-AUC instead of accuracy. The full-dataset and held-out views are kept separate because the full-dataset recall is inflated by the training red events.

The third challenge was feature determinism. DuckDB window functions are sensitive to the ordering of ties. The feature builder uses an explicit ORDER BY with a unique row identifier so that two runs produce the same feature table. The live scorer uses the same feature order, and the model bundle stores the names rather than relying on the booster's positional names.

The fourth challenge was the sequence model. The LSTM autoencoder can assign a high reconstruction error to a rare but benign event, especially a new machine or a new authentication type. The standalone model produced 24,482 false positives on the held-out set. Stacking the reconstruction error as one feature, rather than using it as the final score, lets the classifier learn when the signal is reliable. This is the main reason the final model is a hybrid rather than a pure autoencoder.

The fifth challenge was interface consistency. During verification, the dataset page copy said “six months” while the database span was 58 days. The report uses the verified 58-day value and does not repeat the UI copy. Similarly, an older dashboard image showed 590 caught events and 349 false positives, while the current score file contains 598 and 269 at the BLOCK threshold. The report uses the current score file and the current API, not the stale image.

5.6 Deployment and Execution

The batch pipeline and the live service run from the project root in a Python virtual environment. The batch commands are listed in Appendix B. The live service is started with `python -m live.app` and listens on port 5000. The development bypass is enabled with `LANL_DEV=1`; it must not be used outside a controlled demonstration. The service loads the model bundle, the autoencoder, and the score file at startup and reports readiness through `/api/health`.

The application was exercised with two automated suites. `test_live.py` runs nine API and browser smoke checks. `test_audit.py` runs a 36-check audit of every page and flow, including login, employee access, replay, dataset metrics, search, and console errors. The results are reported in Chapter 6. The test scripts write screenshots to their working directory; the report's screenshots were captured separately from a clean run so that the report does not depend on test artefacts.

CHAPTER 6

TESTING

Testing had two goals. The first was functional correctness: every route, every decision path, and every replay control should behave as specified. The second was evidence integrity: the numbers shown on the dashboard should match the score file and the model bundle, not an older screenshot or a hardcoded template value. The test suites were therefore written against the live HTTP service, not against mocks.

6.1 Testing Strategy

The strategy used three layers. The API layer was tested with direct HTTP requests for health, replay metadata, model metrics, alerts, users, and replay control. The browser layer was tested with Playwright against a real Chromium instance, including page rendering, navigation, console errors, login, employee access, and replay streaming. The data layer was checked by comparing the API values with the DuckDB and Parquet artefacts, including the 702 red events, the 604 users, and the 598/269 BLOCK confusion counts.

A test passes only when the observable behaviour matches the requirement. For example, the model-metrics test does not merely check that the endpoint returns HTTP 200; it checks that the top feature is `log_pair_rank` with a gain of approximately 208,848, which proves that the feature names were zipped to the booster importances correctly. The replay test checks that the emitted count is greater than zero and that the status field is consistent. The browser test checks that every single-page route loads without a console error.

6.2 Test Environment

The tests ran on the development workstation described in Chapter 3. The Flask service listened on `http://localhost:5000` with `LANL_DEV=1`, the scorer reported `scorer_ready: true`, and the score file reported `scores_loaded: true`. Chromium was driven by Playwright in headless mode. The test scripts were copied to a temporary working directory before execution so that their screenshots and logs did not modify the report workspace.

The test data were the same artefacts used for the report: the 29,905,488-event slice, the 702-event label set, the 21-feature model bundle, and the 9,954-event replay window. The tests did not use a reduced fixture, because a reduced fixture would not catch a mismatch in the real score file. The only exception is the smoke test's partial replay, which stops after a few hundred events to keep the run short.

6.3 Test Cases

Table 6.1 lists the functional test cases, the input or action, the expected result, and the observed result. The identifiers are used in the discussion below. All listed cases passed.

Table 6.1: Test cases and results

ID	Test	Expected result	Status
TC-01	Health endpoint	status ok, scorer and scores ready	Pass
TC-02	Replay metadata	window 9,954 events and 20 attacks	Pass
TC-03	Model metrics and feature names	top feature log_pair_rank, gain 208,848	Pass
TC-04	Replay reset, start, status, stop	events emitted, state consistent	Pass
TC-05	Alerts and users during replay	non-empty rows from the replay window	Pass
TC-06	Simulated data quarantine	simulated events do not enter history	Pass
TC-07	Dashboard render	replay controls and event table visible	Pass
TC-08	Live event stream	40 rows appended without reload	Pass
TC-09	All SPA routes	no console errors on any page	Pass
TC-10	Employee normal sign-in	ALLOW decision with a reason	Pass
TC-11	Analyst password plus TOTP	authenticated session and redirect	Pass
TC-12	Dataset evidence page	604 users, 702 red events, confusion counts	Pass
TC-13	Investigation drawer	score, features, history, and label visible	Pass
TC-14	Logout and protected route	session cleared and analyst route protected	Pass

6.4 Automated Test Results

`test_live.py` completed with nine passes and no failures. The health check returned `{'status': 'ok', 'scorer_ready': true, 'scores_loaded': true}`. The replay status reported `window=9954 attacks=20`. The model-metrics check reported `top feature=log_pair_rank gain=208848`. The replay control sequence reset, started, reported a running status, and stopped cleanly. The alerts and users endpoint returned non-empty replay rows. The simulated data check reported zero simulated events in history. The dashboard rendered, the live stream appended 40 rows, and every single-page route loaded without a console error.

`test_audit.py` completed with 36 passes. The audit covered the analyst dashboard, auto-login, the sidebar, the top bar, all seven analyst views plus Settings, the employee login and access decision, the analyst login, the dataset view, the model view, the alerts view, the users view, the search overlay, the investigation drawer, and logout. The audit also captured screenshots for each page. The run reported no failed checks and no uncaught browser exceptions.

The replay validation used the clean final run. It contained 9,954 events, 20 red-team events, and 12 false positives. All 20 attacks were surfaced: 19 as BLOCK and 1 as FLAG. The window also produced 31 BLOCK actions and 1 FLAG action. The 12 false positives are reported as a false-positive count, not hidden inside an accuracy figure. The smoke test's partial run is not used for this result because it stops before the window is complete.

6.5 Observations and Limitations

The functional tests are strong on route coverage and data integrity, but they are not a load test. They do not measure throughput under concurrent analysts, long-running memory growth, or failure recovery after a database lock. They also do not include a penetration test of the session and TOTP implementation. Those are production concerns beyond the scope of a single-workstation research prototype.

The browser audit depends on the development bypass for some flows. The bypass is disabled unless `LANL_DEV=1` is set, and the report does not reproduce the credentials. A production deployment would remove the bypass, store secrets outside the repository, and add rate limiting and audit logging for failed login attempts.

Finally, the tests verify behaviour, not model quality. A passing API test does not prove that a threshold is well chosen or that a feature is causally sound. The model evidence is reported separately in Chapter 7, and the held-out split is the primary defence against an optimistic result. The test suite and the evaluation suite are complementary, not substitutes.

CHAPTER 7

RESULTS AND DISCUSSION

This chapter reports the measured results. It separates the held-out evaluation from the full-dataset evaluation, reports the baseline variants, explains the feature importance, examines the score separation, and validates the model on a real replay window. The discussion focuses on what the numbers do and do not support.

7.1 Experimental Setup

The evaluation uses a user-level GroupShuffleSplit with a 70/30 split and random state 42. The 604 users are divided into 422 training users and 182 test users. The training partition contains 462 red-team events; the test partition contains 240. No user appears in both partitions. The held-out test set has 5,399,886 events, of which 5,399,646 are normal and 240 are attacks. The attack prevalence is therefore 4.4445e-5, or 0.0044%.

The operating threshold is 0.187195. An event at or above the threshold is a BLOCK; an event between 0.131037 and 0.187195 is a FLAG; an event below 0.131037 is an ALLOW. The held-out precision, recall, and F1 in this chapter are computed at the BLOCK threshold. The FLAG band is reported separately because it changes the analyst workload. The full-dataset results include training rows and are labelled as in-sample.

$$\text{Precision} = TP / (TP + FP) \tag{7.1}$$

$$\text{Recall} = TP / (TP + FN) \tag{7.2}$$

$$F1 = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall}) \tag{7.3}$$

$$FPR = FP / (FP + TN) \tag{7.4}$$

$$AP = \text{sum_n } (\text{Recall_n} - \text{Recall_(n-1)}) * \text{Precision_n} \tag{7.5}$$

Equation (7.5) is the step-wise average precision computed from the precision-recall curve. The random baseline is the attack prevalence, 4.4445e-5. The lift is the ratio of the model's average precision to that baseline. ROC-AUC is reported as a secondary summary because it is insensitive to the class prior and can look almost perfect when the negative class dominates.

7.2 Held-Out Detection Results

Table 7.1 reports the held-out confusion matrix and the derived metrics. The model catches 136 of 240 attacks, misses 104, and raises 183 false positives. Precision is 0.4263, recall is 0.5667, and F1 is 0.4866. The false-positive rate is 3.39e-5 because the true-

negative count is very large. PR-AUC is 0.3682, which is 8,284 times the random baseline. ROC-AUC is 0.99992.

Table 7.1: Held-out detection metrics at the BLOCK threshold

Metric	Value	Note
Test events	5,399,886	182 unseen users
Test attacks	240	Held-out red-team events
True positives	136	Attack events at or above 0.187195
False negatives	104	Attack events below the threshold
False positives	183	Normal events at or above the threshold
True negatives	5,399,463	Normal events below the threshold
Precision	0.4263	136 / (136 + 183)
Recall	0.5667	136 / 240
F1	0.4866	Harmonic mean
False positive rate	3.39e-5	183 / 5,399,646
PR-AUC	0.3682	Average precision
ROC-AUC	0.99992	Secondary summary
Random PR baseline	4.44e-5	Attack prevalence
Lift over baseline	8,284x	0.3682 / 4.4445e-5
BLOCK threshold	0.187195	Operating point

The confusion matrix is asymmetric in a way that matters operationally. The model is conservative about BLOCK: it misses 43.3% of held-out attacks, but it also keeps the false-positive count below 200 in 5.4 million events. If the FLAG band is counted as an alert, the model surfaces 147 attacks and 233 false positives, which raises recall to 0.6125 but lowers precision to 0.3868 and F1 to 0.4742. The FLAG band is therefore a recall-oriented triage queue, not a free improvement.

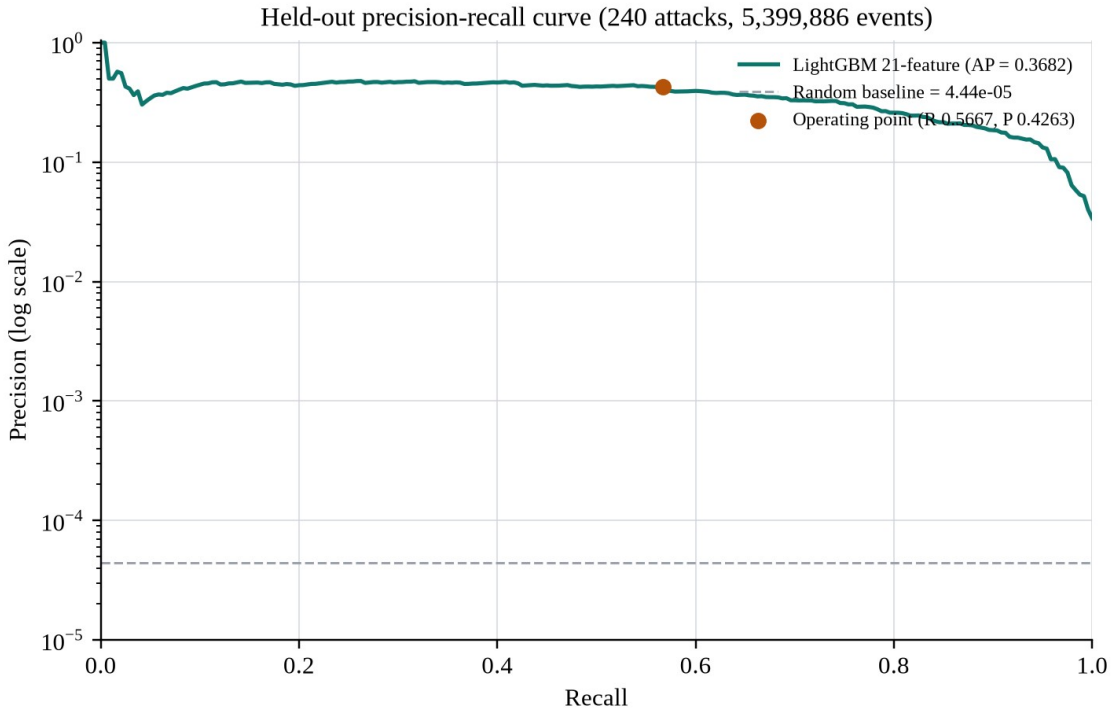


Fig. 7.1: Held-out precision-recall curve

Fig. 7.1 shows the precision-recall curve. The operating point is marked at recall 0.5667 and precision 0.4263. The curve is not smooth at the very low-recall end because only 240 attacks are available; a handful of high-scoring normal events can move the precision substantially. The important region is the middle of the curve, where the model trades recall for precision at a rate that is useful for triage.

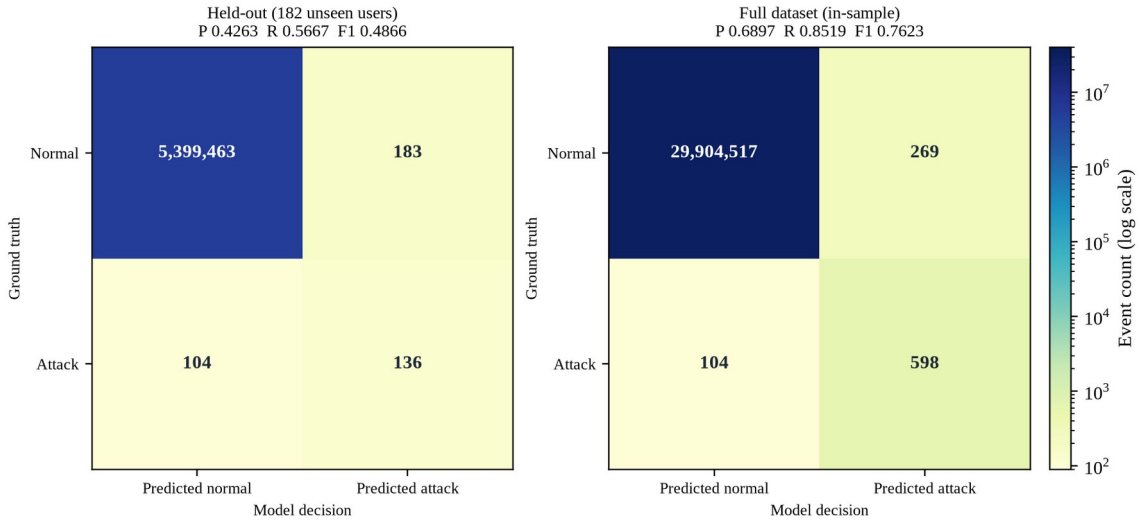


Fig. 7.2: Confusion matrices for held-out and full-dataset evaluation

Fig. 7.2 places the held-out and full-dataset confusion matrices side by side. The visual contrast is the main result of the chapter: the same model that catches 56.7% of unseen attacks catches 85.2% of all attacks when the training rows are included. The difference is not a modelling improvement; it is the effect of evaluating on data the model has already seen.

7.3 Full-Dataset Results

Table 7.2 reports the full-dataset confusion matrix at the BLOCK threshold. The model catches 598 of 702 red-team events, misses 104, and raises 269 false positives. Precision is 0.6897, recall is 0.8519, and F1 is 0.7623. The false-positive rate is 9.0e-6. These are the numbers shown on the Dataset Analysis page when FLAG is not counted as an alert.

Table 7.2: Full-dataset confusion matrix at the BLOCK threshold

	Predicted normal	Predicted BLOCK	Total
Actual normal	29,904,517	269	29,904,786
Actual attack	104	598	702
Total	29,904,621	867	29,905,488

The 104 false negatives are exactly the 104 held-out misses. All 462 training red-team events are caught at the BLOCK threshold. The full-dataset true positives are therefore 462 training detections plus 136 held-out detections. This decomposition is the cleanest

explanation of the optimism gap: the model has memorised the training identities, and the full-dataset recall is not an estimate of performance on a new identity.

Counting FLAG as an alert changes the operational picture. The FLAG band adds 11 red-team events and 103 normal events, so the combined totals become 609 caught, 372 false positives, recall 0.8675, precision 0.6208, and F1 0.7237. The dashboard reports both views. The BLOCK view measures the high-confidence detector; the combined view measures the queue that an analyst would actually review.

7.4 Model and Feature Comparison

Table 7.3 compares the selected 21-feature model with the main variants. RUN 7 uses the 20 behavioural features without the reconstruction error. RUN 8 is the standalone LSTM autoencoder. RUN 10 adds 16 more features, and RUN 11 uses a smoothed autoencoder score. The selected model is RUN 9 because it has the best held-out F1 and the lowest false-positive count among the supervised variants.

Table 7.3: Held-out comparison of model variants

Run	Features	F1	TP	FP	Note
RUN 7	20	0.4817	145	217	No sequence feature
RUN 8	LSTM-AE only	0.0095	118	24,482	Standalone autoencoder
RUN 9	21	0.4866	136	183	Selected model
RUN 10	37	0.4772	157	261	Extra features
RUN 11	23	0.4735	143	221	Smoothed AE score

The comparison is more informative than a single F1. Adding the reconstruction error improves F1 by 0.0049 and reduces false positives by 34, from 217 to 183, but it also reduces true positives by 9, from 145 to 136. Precision rises from 0.4006 to 0.4263. The result is a precision-oriented trade, not a uniform improvement. The standalone autoencoder is not viable: it catches 118 attacks but raises 24,482 false positives, which is why its F1 collapses to 0.0095. RUN 10 and RUN 11 show that adding more features or smoothing the sequence score does not automatically help; both lose F1 relative to RUN 9.

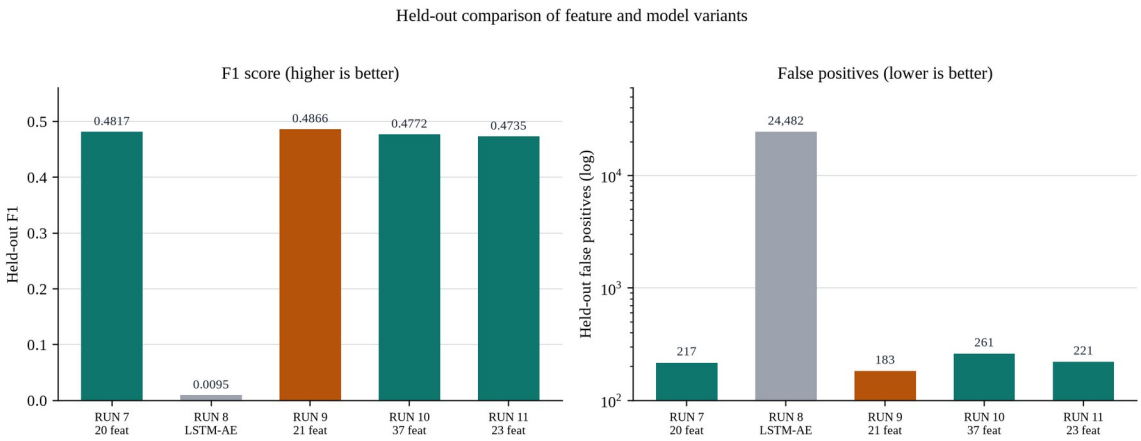


Fig. 7.3: Held-out comparison of feature and model variants

7.5 Feature Importance

Table 7.4 lists the top ten features by LightGBM split gain. The top five account for 84.3% of the total gain: log_pair_rank (40.5%), is_ntlm (26.2%), dst_prior_events (9.6%), pair_freq_ratio (4.6%), and machine_popularity (3.4%). The reconstruction error is rank 8 with 2.4% of the gain. The ranking is consistent with the domain: pair familiarity, authentication protocol, and destination popularity carry most of the signal.

Table 7.4: Top ten features by LightGBM split gain

Rank	Feature	Gain	Share
1	log_pair_rank	208,847.68	40.5%
2	is_ntlm	135,078.67	26.2%
3	dst_prior_events	49,518.82	9.6%
4	pair_freq_ratio	23,659.96	4.6%
5	machine_popularity	17,459.48	3.4%
6	vel_1h	14,645.28	2.8%
7	hour_sin	13,132.48	2.5%
8	lstm_ae_recon_error	12,179.94	2.4%
9	velocity_ratio	8,825.93	1.7%
10	hour_ratio	7,763.03	1.5%

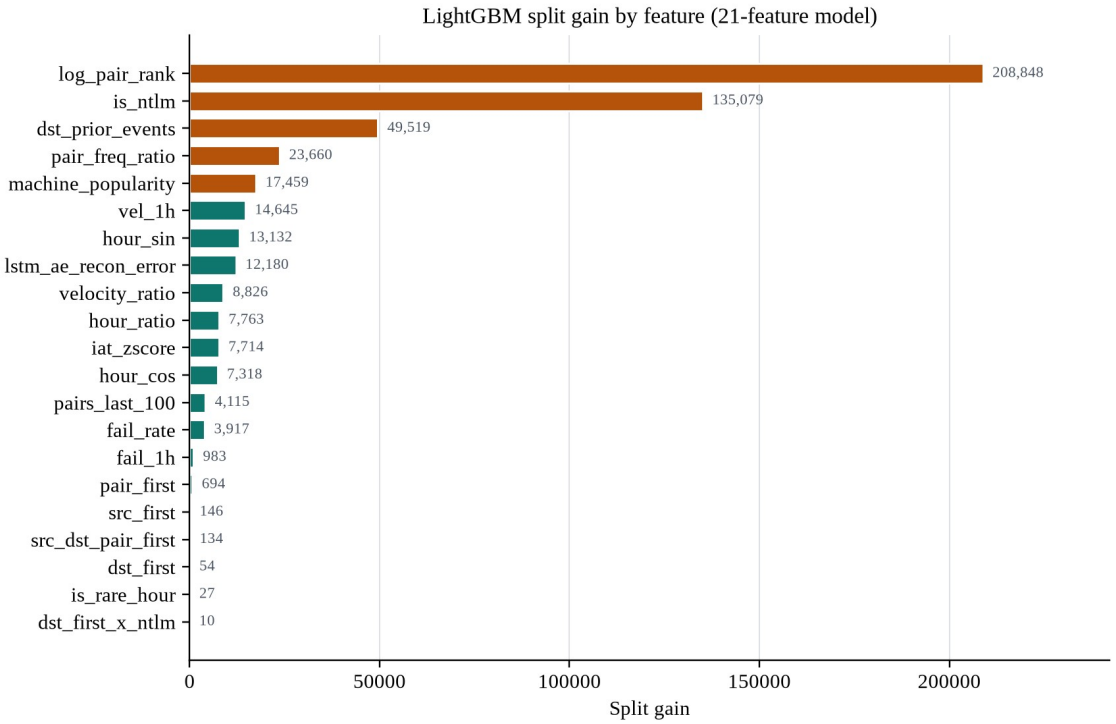


Fig. 7.4: LightGBM split gain by feature

The low rank of the reconstruction error does not mean the sequence model is useless. Split gain measures how often a feature is used to split, not how much it changes the final decision. The ablation in Table 7.3 shows that the feature changes the ranking enough to improve precision and F1. The interpretation is that the reconstruction error is a useful correction on top of a strong tabular model, not the primary detector. This is exactly the role the literature suggests for reconstruction in a hybrid system [5], [6].

7.6 Score Separation

Fig. 7.5 shows the score distribution by class on a logarithmic scale. The separation is strong at the high end: 88.0% of red-team events score at or above 0.1, compared with 0.0015% of normal events. At the 0.01 level, 96.3% of attacks and 0.0066% of normal events are above the line. The overlap is concentrated in the low-score region, which is where the 104 held-out misses and the 183 false positives live.

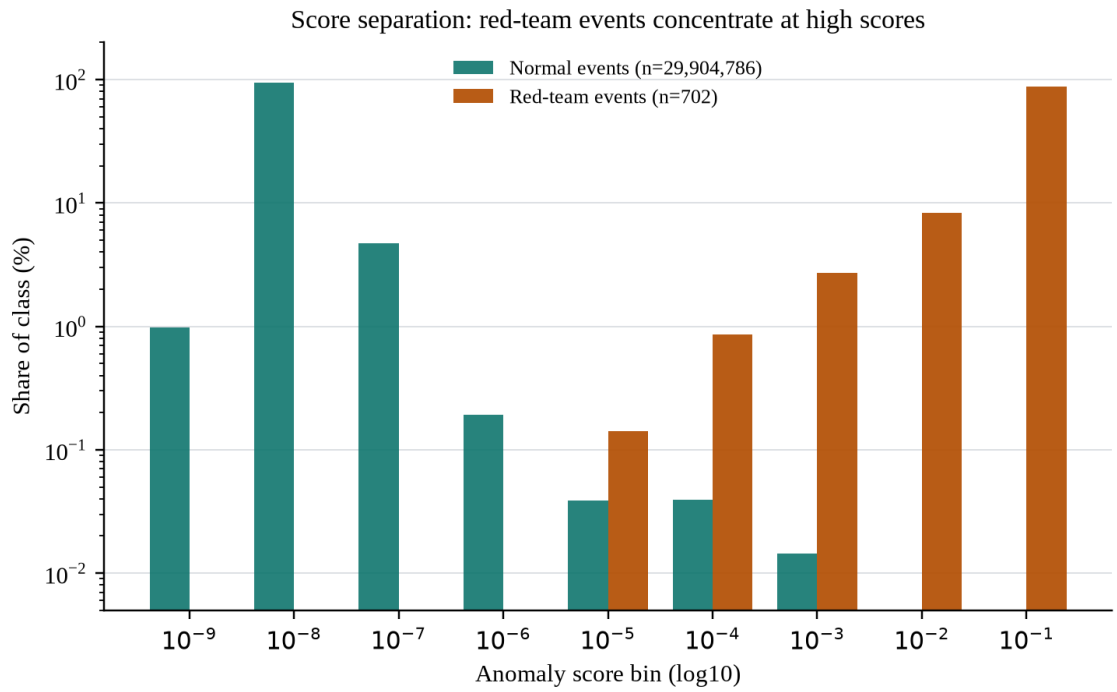


Fig. 7.5: Score separation between normal and red-team events

The shape also explains the precision-recall curve. A large majority of normal events sit at very low scores, so the model can achieve a very low false-positive rate while still missing a meaningful fraction of attacks. The operating threshold is a choice about that trade-off, not a property of the model. Lowering the threshold would catch more attacks and raise the false-positive count; raising it would do the opposite.

7.7 Replay and Live Validation

The replay window is the most realistic test in the project because it preserves the order and the burst structure of a real attack. The window runs from relative time 765000 to 767000, about 33.3 minutes. It contains 9,954 events, 199 users, 200 destinations, and 20 red-team events. The attacker uses source machine C17693 for 34 sign-ins, seven user accounts, and 31 destinations. Table 7.5 reports the outcome.

Table 7.5: Replay-window results

Metric	Value	Note
Window	765000-767000	About 33.3 minutes
Events	9,954	Ordered replay
Users	199	Distinct source users
Destinations	200	Distinct destination machines
Red-team events	20	All from C17693
Surfaced as alert	20	19 BLOCK + 1 FLAG
False positives	12	Normal events in the window
BLOCK actions	31	19 attacks + 12 false positives
FLAG actions	1	1 attack
Largest source	C17693	34 sign-ins, 7 users, 31 destinations

All 20 attacks are surfaced. Nineteen are high-confidence BLOCK decisions and one is a FLAG. The 12 false positives are a reminder that the replay window is attack-dense: the base rate in this 33-minute slice is much higher than the base rate in the full corpus, so the analyst sees more alerts per event than the global false-positive rate would suggest. The window is useful for demonstrating detection and triage, not for estimating daily alert volume.

The live path was also exercised by the test suite. The smoke test streamed 40 rows into the dashboard and the browser audit loaded every page without a console error. Those checks validate the plumbing, not the model quality. The model quality is measured by the held-out and replay results above.

7.8 Discussion

The central result is the gap between the held-out and full-dataset numbers. The held-out model catches 136 of 240 attacks (56.7%) with 183 false positives. The full-dataset model catches 598 of 702 (85.2%) with 269 false positives. The 104 missed attacks are exactly the held-out misses, and all 462 training attacks are caught. This is a textbook in-sample optimism effect. It also means that a report that quotes only 85.2% would overstate the system's ability to generalise to a new identity by a wide margin.

The second result is the value of PR-AUC. ROC-AUC is 0.99992, which sounds almost perfect, but the random PR baseline is only 4.4445e-5 because attacks are extremely rare. The model's PR-AUC of 0.3682 is 8,284 times that baseline, which is a more meaningful statement. At the operating threshold, precision is 0.4263: roughly two of every five BLOCK decisions are attacks. That is useful for triage but not sufficient for automatic enforcement, which is why BLOCK remains a recommendation and not an account lock.

The third result is the modest but real contribution of the sequence model. The reconstruction error is rank 8 by gain, but adding it improves F1 from 0.4817 to 0.4866 and reduces false positives from 217 to 183. The trade is nine fewer true positives. This is not a breakthrough, and the report does not present it as one. It is evidence that a sequence signal can correct a tabular model when it is used as a feature rather than as a standalone detector. The standalone autoencoder result, F1 0.0095 with 24,482 false positives, is the counter-example that makes the point.

The fourth result is the cost asymmetry. The full-dataset false-positive rate is 9.0e-6, which sounds negligible, but it corresponds to 269 false positives. In the replay window, 12 false positives appear in 33 minutes. An analyst does not experience a rate; the analyst

experiences a queue. The BLOCK/FLAG split, and the C-score argument [9], are practical ways to make that queue explicit. A deployment would need to choose the threshold from the local cost of a missed intrusion versus the local cost of an investigation, not from F1 alone.

The fifth result is the boundary of the evidence. The model uses authentication events only. It does not see process starts, DNS, or network flows, which are present in Cyber-1 and which a production SIEM would correlate. The evaluation covers 604 users over 58 days. The red-team behaviour is real, but it is one organisation's red team, and the label table has 13 unmatched tuples. The model has not been tested for concept drift, adversarial evasion, or a different enterprise. Those limits do not invalidate the result; they define the claim. The claim is that a compact behavioural feature set plus a sequence feature can surface a useful fraction of red-team authentication events with a manageable false-positive count, and that an analyst can inspect the evidence in a working dashboard.

Compared with the literature, the result is not directly comparable to Hopper's path-based detection [4] or DabLog's sequence-only detection [5]. The data, the label definition, the features, and the alert budget are different. What is comparable is the design lesson: pair familiarity and protocol features carry most of the signal, sequence reconstruction helps as a correction, and evaluation must be identity-aware. The project's contribution is the combination of those lessons in a reproducible pipeline with an analyst-facing interface.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project built and evaluated an identity anomaly detection system on the LANL Cyber-1 authentication corpus. The system reduces 29.9 million events from 604 users into 21 causal behavioural features, scores each event with a LightGBM classifier, and uses an LSTM autoencoder reconstruction error as the twenty-first feature. It exposes the result through a Flask dashboard with live monitoring, replay, alerts, user views, model evidence, dataset evidence, and TOTP-protected analyst access.

The held-out result is 136 true positives, 183 false positives, 104 false negatives, precision 0.4263, recall 0.5667, F1 0.4866, and PR-AUC 0.3682 on 5,399,886 events from 182 unseen users. The full-dataset result is 598 true positives, 269 false positives, 104 false negatives, precision 0.6897, recall 0.8519, and F1 0.7623. The gap is explained by the user-level split: all 462 training red-team events are caught, and the 104 misses are exactly the held-out misses. The full-dataset number is useful for describing the shipped score file, but it is not a generalisation estimate.

The replay validation surfaces all 20 attacks in the 9,954-event window, with 19 BLOCK and 1 FLAG decisions and 12 false positives. The test suites pass 9/9 smoke checks and 36/36 audit checks. The system is therefore complete as a research prototype: it has a reproducible pipeline, a defensible evaluation, a working interface, and an explicit account of its limits.

8.2 Contributions

- A reproducible 604-user slice and a 21-feature causal feature table derived from the 1.05-billion-event LANL authentication stream.
- A hybrid model that uses an LSTM autoencoder reconstruction error as a tabular feature instead of as a standalone detector, with a measured ablation against the 20-feature baseline.
- A user-level evaluation that exposes the optimism gap between full-dataset and held-out performance, including the exact decomposition of the 104 missed attacks.
- A three-level BLOCK/FLAG/ALLOW decision layer and an analyst dashboard that shows the score, threshold, feature context, history, and ground truth during replay.

- A verified implementation with nine smoke checks, 36 audit checks, and a clean final replay of a real 33-minute attack window.

The four authors carried out the work jointly. The repository does not record per-member ownership, so this report makes a project-level contribution statement and does not assign individual credit that cannot be verified.

8.3 Limitations

- The model uses authentication events only; process, DNS, and network-flow evidence from Cyber-1 is not used.
- The evaluation covers 604 users over 58 days and a red team from one organisation; external validity to another enterprise is unknown.
- The 13 unmatched red-team tuples are excluded rather than guessed, which slightly changes the label set from the raw 715 tuples to 702 events.
- The full-dataset metrics are in-sample and are reported only as a description of the shipped score file, not as a generalisation estimate.
- The live scorer approximates the batch features from precomputed baselines; a production system would need a streaming feature store.
- The development bypass and the local secret storage are suitable for a demonstration, not for an exposed production deployment.
- A UI copy defect said 'six months' while the verified span is 58 days; this report uses 58 days throughout.

8.4 Future Scope

The most valuable next step is to add the other Cyber-1 sources. Process starts, DNS lookups, and network flows can confirm or refute an identity anomaly, and a production detector would correlate them rather than score authentication in isolation. A second step is a streaming feature store that updates the user baselines after every event, so that the live score is exactly the batch score rather than an approximation.

A third step is concept-drift monitoring. The model is trained on 58 days of one network, and the meaning of normal behaviour changes when users change roles, machines are re-imaged, or new applications are deployed. A drift monitor should track the score distribution, the feature distribution, and the alert rate, and trigger retraining when the distributions move beyond a calibrated band. A fourth step is a full SHAP integration [10]

so that the investigation drawer can show signed feature contributions rather than percentile context alone.

A fifth step is cost-aware threshold selection. The BLOCK and FLAG thresholds were chosen from the held-out precision-recall trade-off, not from a measured cost ratio. A deployment should estimate the cost of an investigation and the cost of a missed intrusion, and choose the thresholds that minimise the expected cost [9]. A sixth step is adversarial evaluation: an attacker who knows the feature set may try to stay inside the user's normal pair and timing envelope. Red-team exercises against the live system, rather than against a static score file, would test that failure mode.

Finally, the system should be evaluated with a prospective study. The current results are retrospective: the labels were collected before the model was built, and the replay window is a historical slice. A prospective pilot would run the detector on live authentication events for a fixed period, measure the analyst triage time and the false-positive burden, and compare the alerts with the existing rule-based system. That is the evidence needed before any production claim.

REFERENCES

[1] A. D. Kent, "Cybersecurity Data Sources for Dynamic Network Research," in Dynamic Networks in Cybersecurity. Imperial College Press, 2015.

[2] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," ACM Computing Surveys, vol. 41, no. 3, pp. 1-58, 2009.

[3] M. G\u00fcl and E. Avaro\u011flu, "Multi-Source Behavioral Anomaly Detection for Insider Threat: A Reproducible Empirical Study on the LANL cyber1 Corpus," SSRN Working Paper 7291659, 2026.

[4] G. Ho, M. Dhiman, D. Akhawe, V. Paxson, and D. Wagner, "Hopper: Modeling and Detecting Lateral Movement," in Proc. 30th USENIX Security Symposium, 2021.

[5] L.-P. Yuan, P. Liu, and S. Zhu, "Recomposition vs. Prediction: A Novel Anomaly Detection for Discrete Events Based on Autoencoder," arXiv:2012.13972, 2020.

[6] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection," arXiv:1607.00148, 2016.

[7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 3146-3154.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE International Conference on Data Mining, 2008, pp. 413-422.

[9] M. Marwah, A. Narayanan, S. Jou, M. Arlitt, and M. Pospelova, "Is F1 Score Suboptimal for Cybersecurity Models? Introducing Cscore, a Cost-Aware Alternative for Model Assessment," arXiv:2407.14664, 2024.

[10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 4765-4774.

[11] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, Internet Engineering Task Force, 2011.

APPENDIX A

TWENTY-ONE FEATURES

Table A.1 gives the complete feature set in the order stored in the model bundle. The order is part of the model contract: the live scorer, the batch scorer, and the feature-importance endpoint all use this order. The first twenty features are computed by SQL; the twenty-first is produced by the LSTM autoencoder.

Table A.1: The twenty-one input features

#	Feature	Type	Definition
1	dst_first	Binary	First-ever event from this user to this destination machine
2	src_first	Binary	First-ever event from this source computer
3	hour_ratio	Continuous	Events in this hour divided by the user's total events
4	dst_prior_events	Count	Prior events to this destination, computed causally
5	fail_1h	Count	Failed authentications in the previous hour
6	vel_1h	Count	Authentication velocity in the previous hour
7	hour_sin	Cyclical	Sine encoding of the hour of day
8	hour_cos	Cyclical	Cosine encoding of the hour of day
9	is_ntlm	Binary	Authentication type is NTLM
10	pair_first	Binary	First event for this user-to-machine pair
11	src_dst_pair_first	Binary	First event for this source-to-destination pair
12	fail_rate	Continuous	fail_1h divided by (vel_1h + 1)
13	dst_first_x_ntlm	Interaction	dst_first multiplied by is_ntlm
14	log_pair_rank	Continuous	log1p of the pair's row number for the user
15	pair_freq_ratio	Continuous	Pair events divided by the user's total events
16	is_rare_hour	Binary	Hour is in the bottom 20% of the user's distribution
17	pairs_last_100	Count	Distinct destinations in the last 100 events
18	iat_zscore	Continuous	Inter-arrival time z-score over the last 100 events
19	velocity_ratio	Continuous	vel_1h divided by (events in last 24 hours + 1)
20	machine_popularity	Count	Distinct users accessing the destination machine
21	lstm_ae_recon_error	Continuous	LSTM autoencoder reconstruction error for the event

The first twenty features are computed only from information available at or before the event timestamp. The reconstruction error is computed from a 50-event context that ends at the event. Neither path uses a future event. The deterministic order is time, source user, source computer, destination computer, and row identifier; this is the order that makes the window features reproducible.

APPENDIX B

REPRODUCIBILITY ARTIFACTS

Table B.1 lists the entry points and the artefacts that make the reported results reproducible. The commands require the project virtual environment and the LANL data described in Chapter 3. The repository README contains the exact flags and the data download instructions; the table lists the stage and the output so that a reader can trace each number in the report to a file.

Table B.1: Reproducibility commands and artifacts

Stage	Entry point	Artefact
Slice	src/00_build_slice.py	data/processed/auth_slice.parquet
Features	src/01_build_features.py	data/processed/lanl.duckdb (feat table)
LSTM autoencoder	src/05_lstm_autoencoder.py	models/ lanl_lstm_ae_2ep_bs128_20260913_162002.pt
LightGBM	src/03_retrain_both.py	models/lanl_lgb_21feat.joblib
Batch scores	eval_lstm.py / scoring stage	data/processed/lanl_scores.parquet
Live service	python -m live.app	http://localhost:5000
Smoke tests	test_live.py	9 API and browser checks
Audit tests	live/test_audit.py	36 UI/API checks and screenshots

The score file is approximately 370 MB and contains 29,905,488 rows. The model bundle stores the feature order, the threshold, the held-out metrics, and the hyperparameters. The LSTM checkpoint stores the state dict, the vocabulary, and the field offsets. The DuckDB feature store is read-only at serving time. These four files are sufficient to reproduce the dashboard without retraining.

The four authors carried out the work jointly. The repository does not contain per-member commit attribution for the project, so no individual ownership is asserted here. All four authors reviewed the final report and the accompanying demonstration.